APLICATIVO PARA LA CONVERSIÓN PASO A PASO DE GRAMÁTICAS LIBRES DE CONTEXTO DE LA FORMA NORMAL DE CHOMSKY A LA FORMA NORMAL DE GREIBACH

PRESENTADO POR:
DIEGO ANDRÉS ALVAREZ LIZARAZO
JUAN DAVID QUINTERO MELO
DIEGO ARMANDO MARQUEZ RIGAUD

TEORÍA DE LA COMPUTACIÓN

DOCENTE:
CARMEN JANETH PARADA

UNIVERSIDAD FRANCISCO DE PAULA SANTANDER
FACULTAD DE INGENIERÍA
CÚCUTA, NORTE DE SANTANDER
18 DE OCTUBRE DE 2026

LISTA DE CONTENIDO

INTRODUCCIÓN..... 4
1. PRESENTACIÓN GENERAL DEL PROYECTO..... 8
1.1 TITULO DEL PROYECTO..... 8
1.2 PLANTEAMIENTO DEL PROBLEMA..... 8
1.3 OBJETIVOS..... 11
1.3.1 OBJETIVO GENERAL..... 11
1.3.2 OBJETIVOS ESPECÍFICOS..... 12
2. MARCO TEÓRICO..... 13
3. REQUERIMIENTOS..... 18
3.1 REQUERIMIENTOS FUNCIONALES..... 18
3.2 REQUERIMIENTOS NO FUNCIONALES..... 21
4. ANÁLISIS Y DISEÑO..... 23
5. ALGORITMOS DE CONVERSIÓN..... 28
6. IMPLEMENTACIÓN..... 34
7. PRUEBAS..... 41
8. RESULTADOS..... 64
9. CONCLUSIONES..... 67
10. REFERENCIAS BIBLIOGRÁFICAS..... 70

LISTA DE TABLAS

Tabla 1 Requerimientos funcionales.....19
Tabla 2 Requerimientos no funcionales.....22
Tabla 3 módulos.....35
Tabla 4 componentes.....36
Tabla 5 Pasos FNG.....49
Tabla 6 Resultados de las pruebas..... 65

LISTA DE FIGURAS

Figura 1. Diagrama general de casos de uso.....24
Figura 2. Diagrama de clases.....25
Figura 3. Diagrama de actividades.....26
Figura 4. Diagrama de flujo del algoritmo.....27
Figura 5. Diagrama general del software.....28
Figura 6. Validación FNC..... 43
Figura 7. Ejecución proceso completo.....44
Figura 8. Recursividad inmediata a la izquierda.....46
Figura 9. Recursividad inmediata a la izquierda.....47
Figura 10. Resultado final.....48
Figura 11. Recursividad a la izquierda.....50
Figura 12. Recursividad inmediata a la izquierda.....51
Figura 13. Sustitución final.....52
Figura 14. supera el número de producciones máximo..... 54
Figura 15. Caso A..... 58
Figura 16. Caso B..... 59
Figura 17. Caso C..... 60
Figura 18. Caso D..... 61
Figura 19. Caso E.....62
Figura 20. Caso F.....63
Figura 21. Validación en tiempo real..... 65

INTRODUCCIÓN

Dentro de la Teoría de la Computación, las gramáticas libres de contexto constituyen una de las herramientas formales más utilizadas para describir la estructura de los lenguajes de programación, los protocolos de comunicación y, en general, cualquier lenguaje que pueda definirse mediante reglas de producción. Sobre estas gramáticas existen diversas formas normales representaciones equivalentes pero restringidas en la forma de sus producciones que facilitan tanto su análisis teórico como su procesamiento computacional. Dos de las más estudiadas son la Forma Normal de Chomsky (FNC), en la que toda producción se reduce a dos variables o a un único terminal, y la Forma Normal de Greibach (FNG), en la que toda producción debe iniciar obligatoriamente con un símbolo terminal seguido, opcionalmente, de una cadena de variables. Esta segunda forma resulta particularmente relevante porque garantiza que cada paso de una derivación consuma al menos un símbolo de la cadena generada, propiedad que la hace especialmente útil en la construcción de analizadores sintácticos descendentes y en demostraciones formales sobre las propiedades de los lenguajes libres de contexto.

Convertir una gramática de FNC a FNG, sin embargo, no es un procedimiento trivial. Requiere imponer un orden total sobre las variables, detectar y eliminar toda forma de recursividad izquierda tanto inmediata como indirecta, esta última especialmente difícil de identificar a simple vista cuando involucra ciclos de dependencia entre tres o más variables y aplicar sustituciones sucesivas hasta que la totalidad de las producciones cumpla la forma exigida. Realizado manualmente, este procedimiento es extenso, repetitivo y altamente propenso a errores humanos, incluso para gramáticas de tamaño moderado; un solo paso mal aplicado, o una variable recursiva pasada por alto, invalida por completo el resultado final. Esta dificultad se agrava en el

contexto académico, donde los estudiantes requieren no solo llegar al resultado correcto, sino también comprender y verificar cada una de las transformaciones intermedias que lo producen, algo que el desarrollo a mano en papel rara vez permite hacer de forma clara y sin errores de arrastre.

Frente a esta necesidad, el presente proyecto propone el desarrollo de una herramienta web que automatice por completo el proceso de conversión de FNC a FNG, liberando al usuario de la ejecución manual del algoritmo sin ocultarle su funcionamiento interno. El propósito no es únicamente entregar la gramática final transformada, sino hacer visible y comprensible cada una de las etapas del procedimiento renombrado de variables, eliminación de recursividad, reordenamiento por índices y sustitución final de modo que la aplicación funcione simultáneamente como herramienta de solución y como apoyo pedagógico para el estudio del algoritmo. Se espera que esta herramienta beneficie tanto a quienes cursan la asignatura, al permitirles verificar sus propios desarrollos manuales, como a quienes buscan una referencia confiable y rápida para resolver gramáticas de mayor complejidad.

En consecuencia, el objetivo general de este proyecto es desarrollar una aplicación web que permita convertir automáticamente una gramática en Forma Normal de Chomsky a su equivalente en Forma Normal de Greibach, validando previamente que la gramática de entrada cumpla la forma exigida y verificando al final que el resultado obtenido sea una FNG válida, mediante una arquitectura cliente-servidor que separe la lógica algorítmica de la interfaz de usuario. De este objetivo general se desprenden los objetivos específicos que guiaron el desarrollo: la implementación de la validación formal de la FNC de entrada, la construcción modular de cada uno de los pasos del algoritmo de conversión, el diseño de una interfaz que

permita visualizar el proceso tanto de forma automática como paso a paso, y el despliegue de la herramienta en un entorno accesible públicamente.

Para alcanzar estos objetivos, el proyecto se desarrolló bajo una metodología de construcción incremental, dividiendo el algoritmo de conversión en módulos independientes y verificables por separado antes de integrarlos en el flujo completo. En el aspecto técnico, se empleó Python con el framework Flask para implementar la lógica de conversión como una API REST, y React con Vite y Tailwind CSS para construir la interfaz de usuario, comunicadas entre sí mediante peticiones HTTP en formato JSON. El desarrollo se acompañó de un proceso constante de pruebas sobre gramáticas de complejidad creciente, lo que permitió identificar y corregir oportunamente un problema de crecimiento combinatorio excesivo de producciones en gramáticas con alta recursividad mutua entre variables, antes de la entrega final del sistema.

El alcance del proyecto cubre la conversión completa de gramáticas en FNC a FNG para alfabetos de variables y terminales definidos por el usuario, con validación de entrada y de salida, y con un límite controlado en el número de producciones generadas internamente, orientado a proteger la estabilidad del servicio ante gramáticas de complejidad extrema. Quedan fuera del alcance del proyecto la conversión desde formas gramaticales distintas a la FNC (es decir, la herramienta no normaliza por sí misma una gramática libre de contexto arbitraria a FNC, sino que exige esa forma como punto de partida), así como el análisis sintáctico de cadenas sobre la gramática resultante, que no es un objetivo de esta herramienta.

Para presentar este trabajo de manera ordenada, el documento se organiza de la siguiente forma: tras este capítulo introductorio, se expone el planteamiento del problema y los objetivos, general y específicos, que delimitan el proyecto. A continuación, el marco teórico recoge los conceptos

formales necesarios para comprender tanto el problema como la solución propuesta, seguido de los requerimientos funcionales y no funcionales que debía satisfacer el sistema. El capítulo de análisis y diseño presenta la arquitectura de la solución apoyada en diagramas UML, mientras que el capítulo del algoritmo de conversión describe en detalle cada una de las fases del procedimiento implementado. Posteriormente, el capítulo de implementación documenta las decisiones técnicas de desarrollo lenguaje, módulos, métodos y estructuras de datos, y el capítulo de pruebas presenta la evidencia de su correcto funcionamiento mediante casos reales ejecutados contra el sistema, incluyendo escenarios límite diseñados para verificar su robustez. El documento cierra con los capítulos de resultados y conclusiones, en los que se evalúa el cumplimiento de los objetivos planteados y se reflexiona sobre los aprendizajes obtenidos durante el desarrollo del proyecto.

1. PRESENTACIÓN GENERAL DEL PROYECTO

1.1 TITULO DEL PROYECTO

Aplicativo Para La Conversión Paso A Paso De Gramáticas Libres De Contexto De La Forma Normal De Chomsky A La Forma Normal De Greibach.

1.2 PLANTEAMIENTO DEL PROBLEMA

Las gramáticas libres de contexto no solo sirven para definir un lenguaje: la forma específica en que están escritas sus producciones determina qué algoritmos pueden aplicarse sobre ellas y con qué eficiencia. Así como la Forma Normal de Chomsky (FNC) es la forma que exigen algoritmos de análisis sintáctico ascendente como CYK, la Forma Normal de Greibach (FNG) cumple un papel equivalente para otro tipo de construcciones: toda producción que comienza obligatoriamente con un símbolo terminal, seguido únicamente de variables, permite construir de forma directa un autómata de pila sin transiciones épsilon y facilita el análisis sintáctico descendente, además de ser la base teórica para demostrar que todo lenguaje libre de contexto puede generarse sin recursividad izquierda. Autores de referencia en teoría de lenguajes formales, como Hopcroft y Ullman, señalan que esta normalización no es un simple ejercicio formal, sino una condición estructural necesaria para que ciertas construcciones y demostraciones sobre lenguajes libres de contexto resulten manejables.

Sin embargo, convertir una gramática ya depurada y normalizada en FNC a su forma equivalente en FNG es, en la práctica, un procedimiento considerablemente más delicado que la normalización a Chomsky. No basta con reorganizar símbolos: es necesario establecer primero un orden total entre las variables, detectar y sustituir las producciones en las que una variable

deriva en otra de menor índice, y el paso que con más frecuencia induce al error identificar y eliminar la recursividad izquierda, tanto la inmediata (una variable que se deriva directamente en sí misma) como la indirecta (un ciclo de recursividad que involucra a dos o más variables de forma transitiva). Cada una de estas operaciones exige crear variables auxiliares, sustituir producciones completas por las de otra variable ya resuelta, y repetir ese proceso de manera consistente hasta que absolutamente todas las producciones de la gramática inicien con un símbolo terminal.

El escenario específico donde se sitúa este proyecto es el de un estudiante de ingeniería que, durante un periodo académico, debe resolver a mano, sobre el papel, este tipo de transformación a partir de una gramática ya construida en FNC. Ahí aparece una brecha clara entre el procedimiento ideal y lo que ocurre en la práctica. Idealmente, el estudiante debería poder ordenar las variables, sustituir cada producción problemática y eliminar toda recursividad izquierda sin perder de vista el estado completo de la gramática en cada momento. En la realidad, es común que el proceso se pierda a mitad de camino: una sustitución mal aplicada en una variable puede arrastrar el error a todas las variables que dependan de ella, el orden en que debe resolverse la recursividad indirecta frente a la inmediata no siempre es evidente, y una variable auxiliar mal nombrada o mal vinculada puede hacer que la gramática final, aunque luzca correcta a primera vista, no sea en realidad equivalente a la original ni cumpla con la Forma Normal de Greibach.

Varias causas explican por qué este procedimiento es particularmente propenso al error cuando se ejecuta manualmente. En primer lugar, la naturaleza combinatoria de la sustitución:

reemplazar una sola producción que empieza en una variable de menor índice puede requerir generar tantas producciones nuevas como alternativas tenga esa variable, y ese número crece rápidamente cuando varias variables están mutuamente relacionadas. En segundo lugar, la interdependencia entre pasos: la eliminación de recursividad indirecta debe completarse en toda la gramática antes de abordar la recursividad inmediata, y alterar ese orden o aplicarlo de forma parcial invalida el resultado sin que el error sea evidente de inmediato. En tercer lugar, la ausencia de retroalimentación intermedia: al resolverse completamente a mano, el estudiante solo puede confirmar si el procedimiento fue correcto hasta llegar al final, y para entonces resulta difícil identificar en qué paso específico renombrado, sustitución, eliminación de recursividad o conversión final se originó la desviación.

Las consecuencias de esta dificultad no son menores. Un error de procedimiento, no de comprensión conceptual, puede llevar a que un estudiante entregue una gramática que, pese a haber razonado correctamente la teoría detrás de la Forma Normal de Greibach, no sea equivalente a la gramática original o no cumpla formalmente con dicha normalización. Esto se traduce en calificaciones que no reflejan el verdadero nivel de comprensión del estudiante, en tiempo invertido persiguiendo un error sin saber dónde se originó, y en una consolidación débil del razonamiento algorítmico que este ejercicio busca precisamente afianzar.

Frente a esta situación, resulta pertinente contar con una herramienta que reproduzca fielmente el mismo procedimiento que se exige realizar a mano con la misma notación, el mismo orden de fases y el mismo tratamiento de los casos especiales que suelen generar confusión, como la recursividad indirecta entre varias variables a la vez, pero que lo ejecute de forma

automática y verificable en cada uno de sus pasos. Una solución de este tipo no reemplaza el ejercicio de razonar la gramática; le da al estudiante, y a cualquier persona que esté aprendiendo este tema, un punto de referencia inmediato y confiable contra el cual contrastar su propio trabajo, detectar en qué paso específico se desvió, y reforzar con mayor seguridad el dominio del procedimiento completo de conversión de FNC a FNG.

A partir de lo anterior, este proyecto se propone responder la siguiente pregunta: ¿De qué manera la ausencia de una herramienta que reproduzca, paso a paso y con retroalimentación inmediata, el procedimiento de conversión de una gramática libre de contexto de su Forma Normal de Chomsky a su Forma Normal de Greibach, dificulta que un estudiante verifique y comprenda cabalmente dicho procedimiento?

1.3 OBJETIVOS

1.3.1 OBJETIVO GENERAL

Desarrollar un aplicativo web que convierta de forma automática y paso a paso gramáticas libres de contexto en Forma Normal de Chomsky a su equivalente en Forma Normal de Greibach, con el fin de que el estudiante pueda verificar y comprender el procedimiento de conversión, mediante un backend en Python/Flask y una interfaz web en React.

1.3.2 OBJETIVOS ESPECÍFICOS

- Implementar un módulo de validación que verifique que la gramática ingresada cumpla con la Forma Normal de Chomsky, con el fin de garantizar que el proceso de conversión parta siempre de una entrada estructuralmente correcta.
- Desarrollar un algoritmo de renombrado de variables a notación de Greibach (X1, X2, ..., Xn), con el fin de establecer el orden total entre variables que exige el procedimiento de conversión.
- Desarrollar un algoritmo de detección y eliminación de recursividad indirecta a la izquierda mediante el cálculo de componentes fuertemente conexas entre variables, con el fin de resolver los ciclos de recursividad mutua antes de abordar cualquier otro tipo de transformación.
- Desarrollar un algoritmo de detección y eliminación de recursividad inmediata a la izquierda mediante la creación de variables auxiliares, con el fin de eliminar las producciones en que una variable deriva directamente en sí misma sin alterar el lenguaje generado por la gramática.
- Implementar un algoritmo de sustitución que elimine las producciones en que una variable deriva en otra de menor índice, con el fin de que toda producción quede expresada únicamente en función de variables de índice mayor o de símbolos terminales.
- Implementar un algoritmo de sustitución final que reemplace de manera descendente las producciones restantes, con el fin de que toda producción de la gramática resultante inicie obligatoriamente con un símbolo terminal, cumpliendo así la Forma Normal de Greibach.

- Diseñar una interfaz gráfica que muestre el historial completo de transformaciones y permita avanzar paso a paso por cada etapa de la conversión, con el fin de que el usuario pueda contrastar su propio procedimiento manual contra el resultado generado por el sistema.
- Implementar un generador de ejercicios aleatorios en distintos niveles de dificultad, con el fin de que el estudiante pueda practicar el procedimiento de conversión con gramáticas nuevas en cada intento.
- Desplegar el aplicativo en un entorno de producción accesible desde el navegador, con el fin de que el sistema pueda ser utilizado y sustentado sin depender de una instalación local.

2. MARCO TEÓRICO

Lenguaje formal

Un lenguaje formal es un conjunto de cadenas (secuencias finitas de símbolos) construidas a partir de un alfabeto finito, definido mediante reglas precisas y no ambiguas. A diferencia del lenguaje natural, un lenguaje formal no admite interpretaciones subjetivas: una cadena pertenece o no pertenece al lenguaje según se cumplan o no las reglas que lo definen. Hopcroft, Motwani y Ullman (2006) señalan que el estudio de los lenguajes formales constituye la base teórica sobre la cual se construyen los compiladores, los intérpretes y, en general, cualquier sistema que deba reconocer o generar cadenas siguiendo una estructura determinada. En este proyecto, el lenguaje formal de interés es el conjunto de cadenas que una gramática libre de contexto es capaz de generar.

Gramática formal

Una gramática formal es un mecanismo generativo que describe, mediante un conjunto finito de reglas, cómo construir las cadenas que pertenecen a un lenguaje. Formalmente se define como una tupla $G = (V, T, S, P)$, donde V es el conjunto de variables (o símbolos no terminales), T es el conjunto de símbolos terminales, $S \in V$ es el símbolo inicial, y P es el conjunto de producciones o reglas de reescritura. La gramática no enumera las cadenas del lenguaje una por una, sino que establece el procedimiento mediante el cual cualquier cadena válida puede derivarse a partir del símbolo inicial.

Gramática Libre de Contexto

Una Gramática Libre de Contexto (GLC) es una gramática formal en la que toda producción tiene exactamente una variable en el lado izquierdo, es decir, producciones de la forma $A \rightarrow \alpha$, donde $A \in V$ y α es una cadena cualquiera de símbolos terminales y/o variables (incluyendo la cadena vacía). La restricción de tener una única variable como antecedente implica que una producción puede aplicarse sin importar el contexto los símbolos que la rodean en el que aparezca la variable, de donde proviene su nombre. Las GLC son suficientemente expresivas para describir la sintaxis de la mayoría de los lenguajes de programación y, a la vez, suficientemente restrictivas como para admitir algoritmos de análisis sintáctico eficientes, razón por la cual son el objeto de estudio central de este proyecto.

Símbolos terminales

Los símbolos terminales son los elementos básicos e indivisibles que conforman las cadenas finales de un lenguaje; representan los “caracteres” reales que aparecen en una cadena generada por la gramática y no pueden ser sustituidos por ninguna producción, ya que ninguna regla los tiene como antecedente. En la notación convencional se representan con letras minúsculas o dígitos (a, b, 1, 2, ...). En el aplicativo desarrollado, los terminales declarados por el

usuario son precisamente los símbolos que, al finalizar la conversión a Forma Normal de Greibach, deben aparecer obligatoriamente como primer símbolo de cada producción.

Símbolos no terminales

Los símbolos no terminales, también llamados variables, representan categorías sintácticas o conceptos abstractos del lenguaje que aún pueden expandirse mediante alguna producción de la gramática. Se escriben convencionalmente con letras mayúsculas (A, B, S, X1, X2, ...) y constituyen los elementos que efectivamente se transforman durante el proceso de derivación, hasta que la cadena resultante queda compuesta únicamente por símbolos terminales. El símbolo inicial S es, en particular, la variable a partir de la cual se genera cualquier cadena del lenguaje.

Producciones

Una producción es una regla de reescritura de la forma $A \rightarrow \alpha$ que indica que, en cualquier punto de una derivación, la variable A puede sustituirse por la cadena α. El conjunto completo de producciones P define, junto con el símbolo inicial, todas las cadenas que la gramática es capaz de generar. Una misma variable puede tener varias producciones alternativas, usualmente separadas por el símbolo "|" (por ejemplo, $A \rightarrow aB \mid bC \mid c$), lo que significa que cualquiera de esas alternativas es una sustitución válida para A.

Derivación

La derivación es el proceso de aplicar sucesivamente producciones a partir del símbolo inicial hasta obtener una cadena compuesta únicamente por símbolos terminales. Se dice que una cadena w pertenece al lenguaje generado por la gramática G si existe una secuencia de derivaciones $S \Rightarrow \alpha1 \Rightarrow \alpha2 \Rightarrow ... \Rightarrow w$. Cada paso de la derivación sustituye una variable presente en la cadena actual por el lado derecho de alguna de sus producciones. El concepto de

derivación es el que permite verificar formalmente que dos gramáticas distintas generan exactamente el mismo lenguaje, aun cuando sus reglas de producción luzcan completamente diferentes.

Gramáticas equivalentes

Dos gramáticas son equivalentes cuando generan exactamente el mismo lenguaje, es decir, el mismo conjunto de cadenas, independientemente de que sus conjuntos de variables, terminales o producciones sean distintos entre sí. Todo el proceso de normalización que implementa este proyecto el renombrado de variables, la eliminación de recursividad izquierda y las sucesivas sustituciones de producciones debe preservar en todo momento la equivalencia entre la gramática original en Forma Normal de Chomsky y la gramática final en Forma Normal de Greibach: ambas deben generar el mismo lenguaje, aunque su apariencia sea completamente distinta.

Forma Normal de Chomsky

Una gramática libre de contexto está en Forma Normal de Chomsky (FNC) cuando todas sus producciones tienen una de las siguientes formas: $A \rightarrow BC$ (una variable deriva en exactamente dos variables) o $A \rightarrow a$ (una variable deriva en exactamente un terminal), sin permitir producciones unitarias ($A \rightarrow B$) ni producciones vacías ($A \rightarrow \varepsilon$), salvo el caso particular del símbolo inicial. Esta forma normal es la condición de entrada exigida por este proyecto: el aplicativo asume y valida que la gramática ingresada por el usuario ya se encuentra en FNC, de acuerdo con los lineamientos del microproyecto, que además admiten de manera flexible cualquier combinación de dos símbolos (terminal-terminal, terminal-variable, variable-terminal o variable-variable) en el lado derecho de una producción binaria.

Forma Normal de Greibach

Una gramática libre de contexto está en Forma Normal de Greibach (FNG) cuando toda producción tiene la forma A → aA1A2...An, donde a es un único símbolo terminal y A1, A2, ..., An son cero o más variables. Es decir, toda producción debe iniciar obligatoriamente con un símbolo terminal, seguido únicamente de variables (nunca de otro terminal). Esta forma normal garantiza que, en cualquier derivación, cada paso consuma exactamente un símbolo terminal de la cadena de salida, propiedad que resulta fundamental para construir autómatas de pila sin transiciones épsilon y para ciertas demostraciones sobre las propiedades de los lenguajes libres de contexto. Convertir una gramática de FNC a FNG es precisamente el objetivo central de este proyecto.

Recursividad izquierda

Se dice que una variable presenta recursividad izquierda cuando puede derivarse, directa o indirectamente, en una cadena que comienza por ella misma. La recursividad izquierda inmediata ocurre cuando una producción tiene la forma A → Aα (la variable se deriva directamente en sí misma seguida de alguna cadena). La recursividad izquierda indirecta ocurre cuando el ciclo involucra a dos o más variables de forma transitiva por ejemplo, A deriva en B, y B a su vez deriva en A, de modo que el ciclo solo es detectable analizando las relaciones de dependencia entre varias variables a la vez. Ninguna forma de recursividad izquierda es compatible con la Forma Normal de Greibach, ya que toda producción recursiva por la izquierda necesariamente vuelve a derivar en la misma variable antes de alcanzar un símbolo terminal, por lo que su eliminación es un paso obligatorio del proceso de conversión.

Sustitución de producciones

La sustitución de producciones es la operación mediante la cual, dada una producción A → Bα en la que B es otra variable, se reemplaza esa producción por una nueva para cada alternativa que B pueda derivar: si B → β1 | β2 | ... | βk, entonces A → Bα se sustituye por A → β1α | β2α | ... | βkα. Esta operación es la herramienta fundamental sobre la que se apoyan tanto la eliminación de recursividad izquierda como el ordenamiento de variables y la conversión final a FNG, y debe aplicarse de manera consistente para no alterar el lenguaje generado por la gramática.

Variables auxiliares

Las variables auxiliares son nuevas variables, no presentes en la gramática original, que se introducen durante el proceso de normalización para representar de forma equivalente una recursividad eliminada. En el caso de la eliminación de recursividad inmediata, dada una variable A con producciones recursivas A → Aα1 | ... | Aαm y no recursivas A → β1 | ... | βn, se introduce una variable auxiliar A' tal que A se reescribe como A → β1A' | ... | βnA' | β1 | ... | βn, y A' se define como A' → α1A' | ... | αmA' | α1 | ... | αm. Estas variables auxiliares deben recibir nombres únicos dentro de la gramática para no generar ambigüedad con las variables ya existentes.

3. REQUERIMIENTOS

3.1 REQUERIMIENTOS FUNCIONALES

Tabla 1

Requerimientos funcionales

CÓDIGO	REQUISITO	REQUERIMIENTO
RF01	Ingresar la gramática	El sistema debe permitir registrar las variables (Vnt), los terminales (Vt), el símbolo inicial y las producciones de una gramática libre de contexto.
RF02	Identificar variables y terminales	El sistema debe distinguir, a partir de lo ingresado, cuáles símbolos corresponden a variables y cuáles a terminales.
RF03	Registrar el símbolo inicial	El sistema debe asociar explícitamente un símbolo inicial a la gramática ingresada.
RF04	Registrar las producciones	El sistema debe permitir declarar, para cada variable, una o varias producciones alternativas.
RF05	Validar la Forma Normal de Chomsky	El sistema debe verificar que la gramática ingresada cumpla con la FNC (producciones de la forma VntVnt, VtVt, VtVnt, VntVt o Vt), rechazando producciones unitarias, vacías, con símbolos no declarados o con más de dos símbolos, e indicando el motivo exacto del error.

RF06	Renombrar variables a notación de Greibach	El sistema debe renombrar las variables de la gramática a notación X1, X2, ..., Xn, ubicando el símbolo inicial como X1, para establecer el orden total requerido por el algoritmo.
RF07	Eliminar recursividad indirecta	El sistema debe detectar los ciclos de recursividad mutua entre dos o más variables mediante el cálculo de componentes fuertemente conexas, y sustituirlos hasta eliminarlos por completo.
RF08	Eliminar recursividad inmediata	El sistema debe detectar las producciones en que una variable deriva directamente en sí misma y eliminarlas mediante la creación de variables auxiliares.
RF09	Ordenar producciones por índice	El sistema debe sustituir toda producción $Xi \rightarrow Xj\alpha$ en la que j sea menor que i, hasta que todas las producciones respeten el orden de índices.
RF10	Realizar la sustitución final a FNG	El sistema debe sustituir, de la última variable a la primera, toda producción que aún inicie en variable, hasta que todas las producciones de la gramática inicien en un símbolo terminal.
RF11	Mostrar cada transformación realizada	El sistema debe presentar, para cada paso, las producciones antes y después del cambio, señalando cuáles fueron eliminadas y cuáles son nuevas.
RF12	Mostrar la gramática resultante de cada etapa	El sistema debe presentar la gramática completa resultante al final de cada uno de los

		cinco pasos de conversión.
RF13	Validar la gramática final en FNG	El sistema debe verificar automáticamente que la gramática resultante cumpla con la Forma Normal de Greibach.
RF14	Generar un historial completo	El sistema debe conservar y presentar el registro íntegro de todas las transformaciones realizadas durante la conversión.
RF15	Ofrecer un modo de ejecución paso a paso	El sistema debe permitir al usuario avanzar manualmente paso por paso, revelando únicamente las transformaciones hasta el paso seleccionado.
RF16	Ofrecer un modo de ejecución automática	El sistema debe permitir ejecutar y revelar el proceso completo de conversión de una sola vez.
RF17	Generar ejercicios aleatorios	El sistema debe poder generar gramáticas aleatorias válidas en FNC, en distintos niveles de dificultad, para que el usuario practique el procedimiento de conversión.

Nota. Elaboración Propia

3.2 REQUERIMIENTOS NO FUNCIONALES

Tabla 2

Requerimientos no funcionales

CÓDIGO	REQUISITO	REQUERIMIENTO
RFN01	Interfaz clara y comprensible	La interfaz debe presentar la información mediante una notación formal reconocible y un código de colores consistente (producciones nuevas, eliminadas y normales) que facilite seguir el procedimiento.
RFN02	Aceptar diferentes gramáticas	El sistema debe procesar cualquier gramática válida en FNC ingresada por el usuario, sin limitarse a un conjunto fijo de ejemplos.
RFN03	No depender de casos específicos	Los algoritmos deben implementarse de forma general, operando sobre la estructura de la gramática y no sobre valores particulares codificados.
RFN04	Código organizado y documentado	El código fuente debe estar dividido en módulos independientes según su responsabilidad (validación, renombrado, recursividad, orden de índices, sustitución final, historial, generación de ejercicios) y debe incluir comentarios explicativos.
RFN05	Nombres descriptivos	Las variables, funciones y módulos del código deben tener nombres que reflejen claramente su propósito.

RFN06	Preservar la equivalencia de la gramática	Toda sustitución o eliminación de recursividad debe garantizar que la gramática resultante genere exactamente el mismo lenguaje que la gramática original.
RFN07	Evitar duplicación innecesaria de producciones	El sistema debe deduplicar las producciones generadas durante cada paso, de modo que una misma combinación de cabeza y cuerpo no se repita en la gramática resultante.
RFN08	Nombres únicos para variables auxiliares	Cada variable auxiliar creada durante la eliminación de recursividad inmediata debe recibir un nombre que no colisione con ninguna variable ya existente en la gramática.
RFN09	Registro de cada operación	Cada sustitución o eliminación realizada debe quedar registrada como un evento individual dentro del historial de transformaciones.
RFN10	Resultado reproducible	Ante una misma gramática de entrada, el sistema debe producir siempre la misma secuencia de pasos y el mismo resultado final.
RFN11	Mensajes de error comprensibles	Los errores de validación deben identificarse con un código específico y un mensaje en lenguaje natural que explique la causa exacta del error.
RFN12	Validación automática del resultado final	El sistema debe verificar por sí mismo, sin intervención del usuario, que la gramática final obtenida cumpla

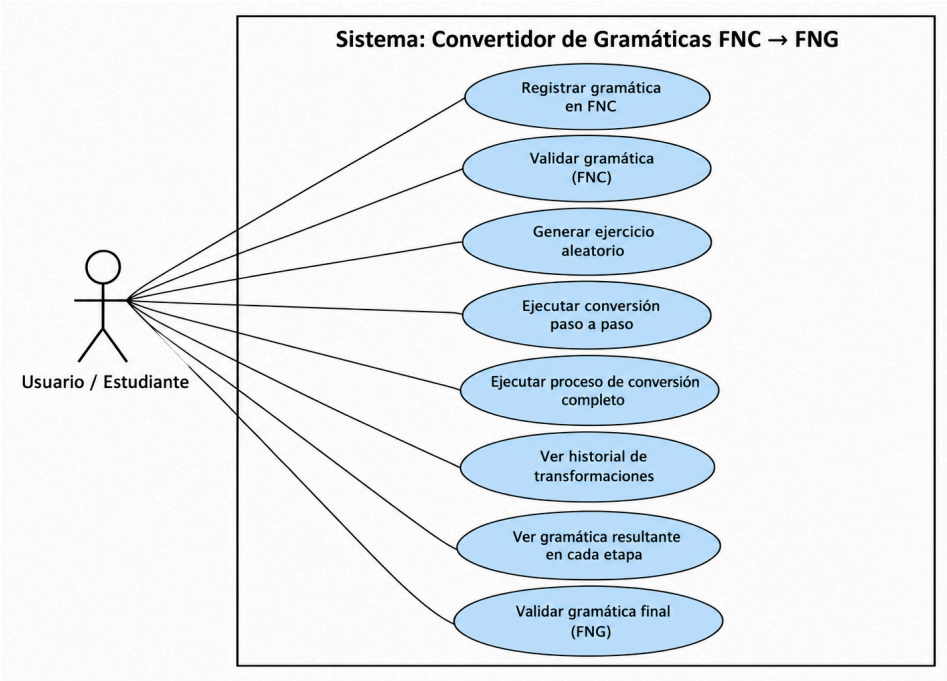
		formalmente con la Forma Normal de Greibach.

Nota. Elaboración Propia

4. ANÁLISIS Y DISEÑO

CASOS DE USO

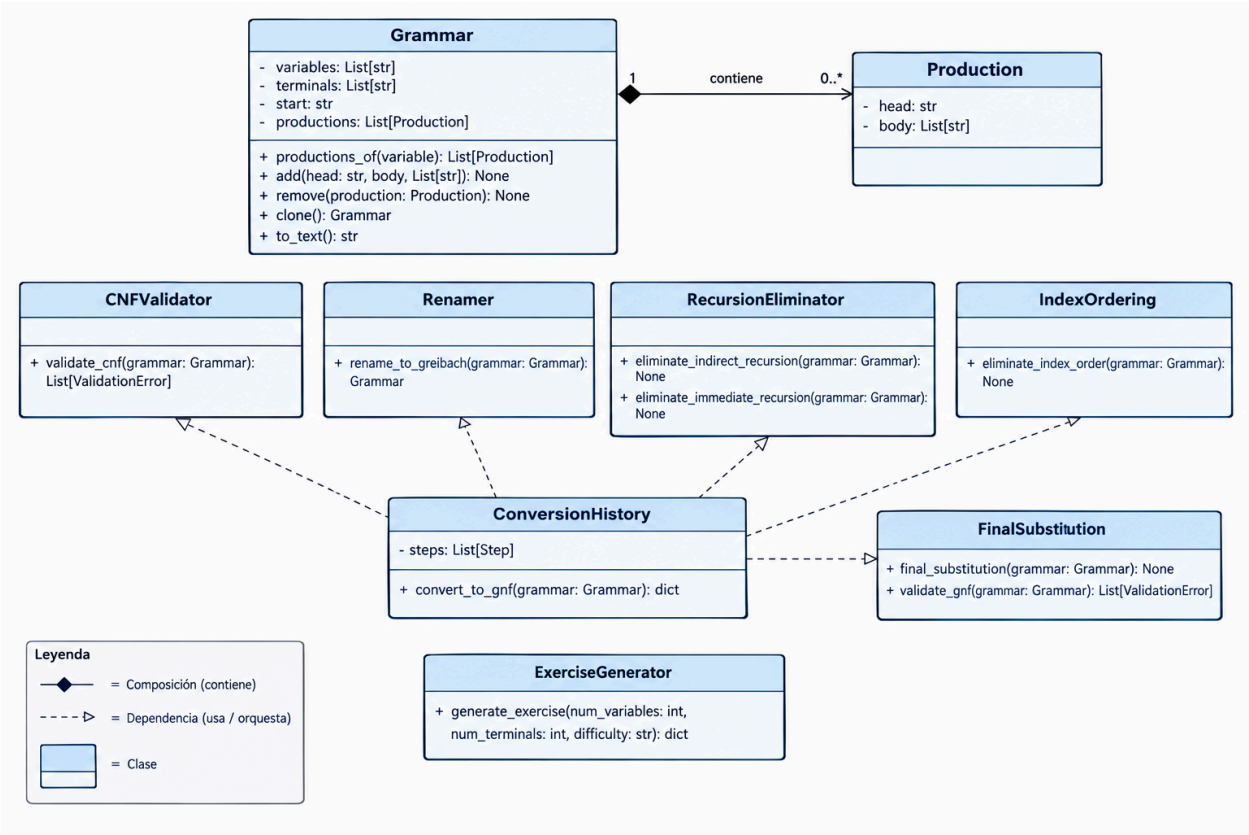
Figura 1. Diagrama general de casos de uso



Nota. Elaborado con IA

DIAGRAMA DE CLASES

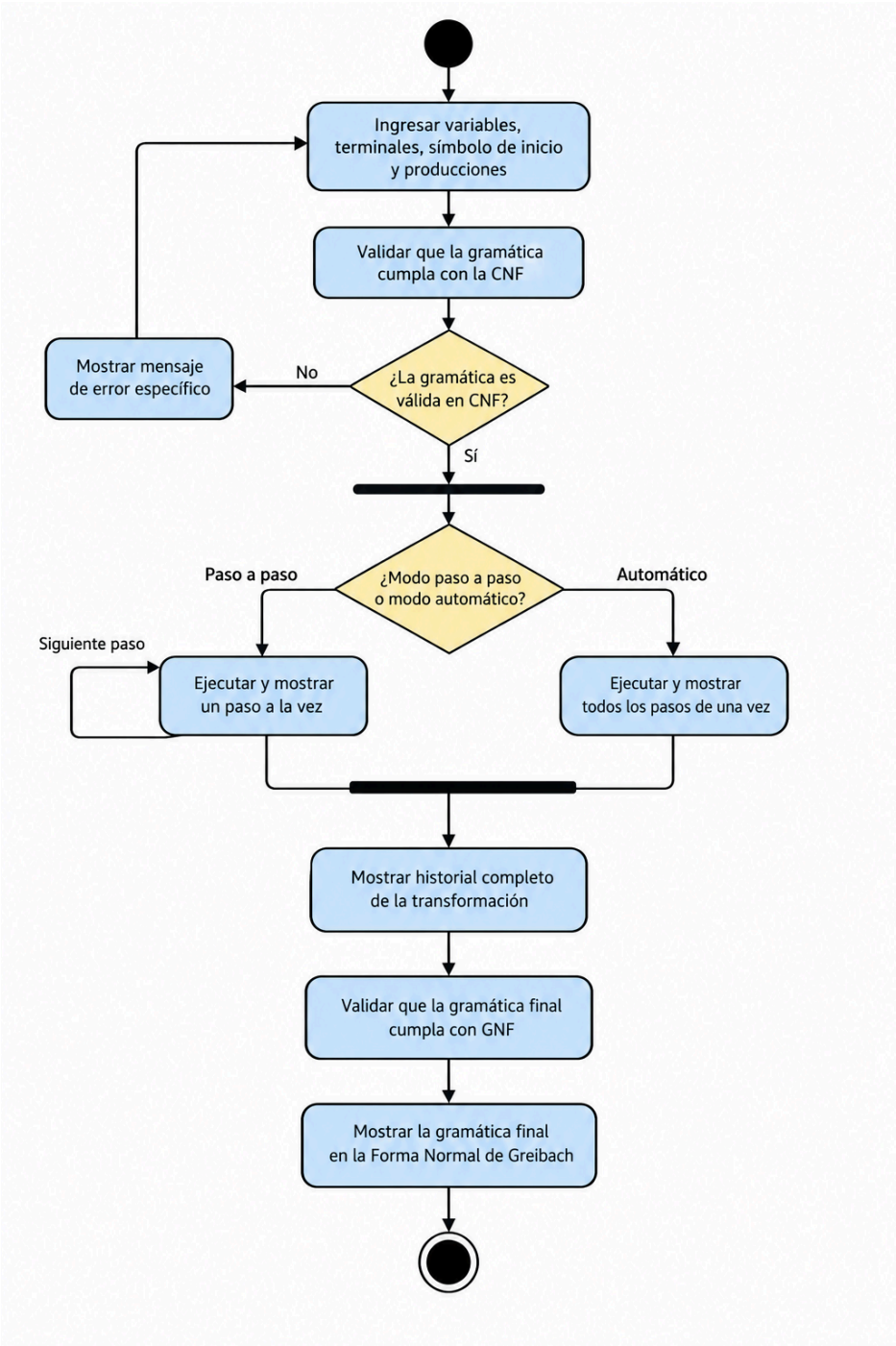
Figura 2. Diagrama de clases



Nota. Elaborado con IA

DIAGRAMA DE ACTIVIDADES

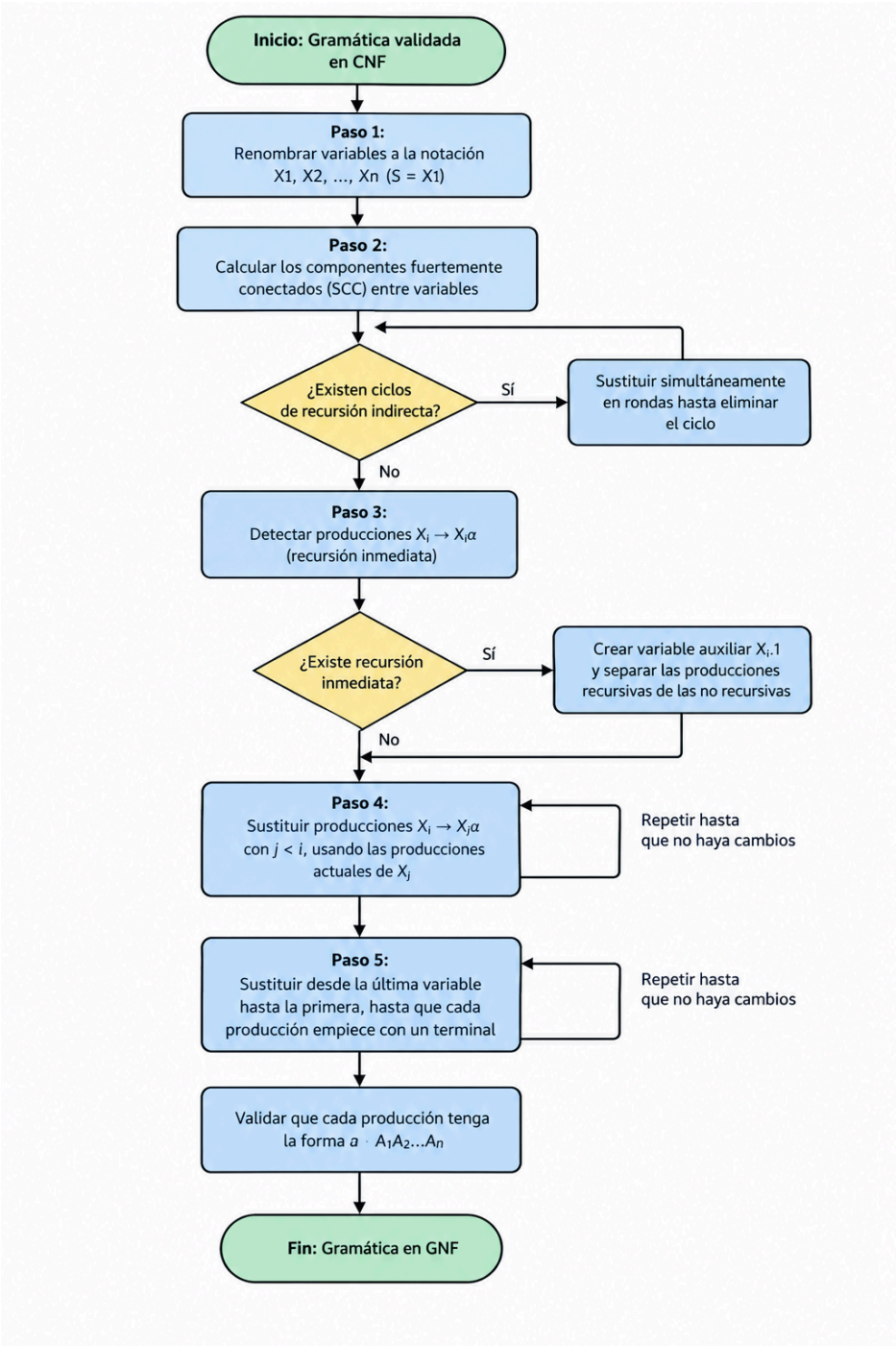
Figura 3. Diagrama de actividades



Nota. Elaborado con IA

DIAGRAMA DE FLUJO DEL ALGORITMO

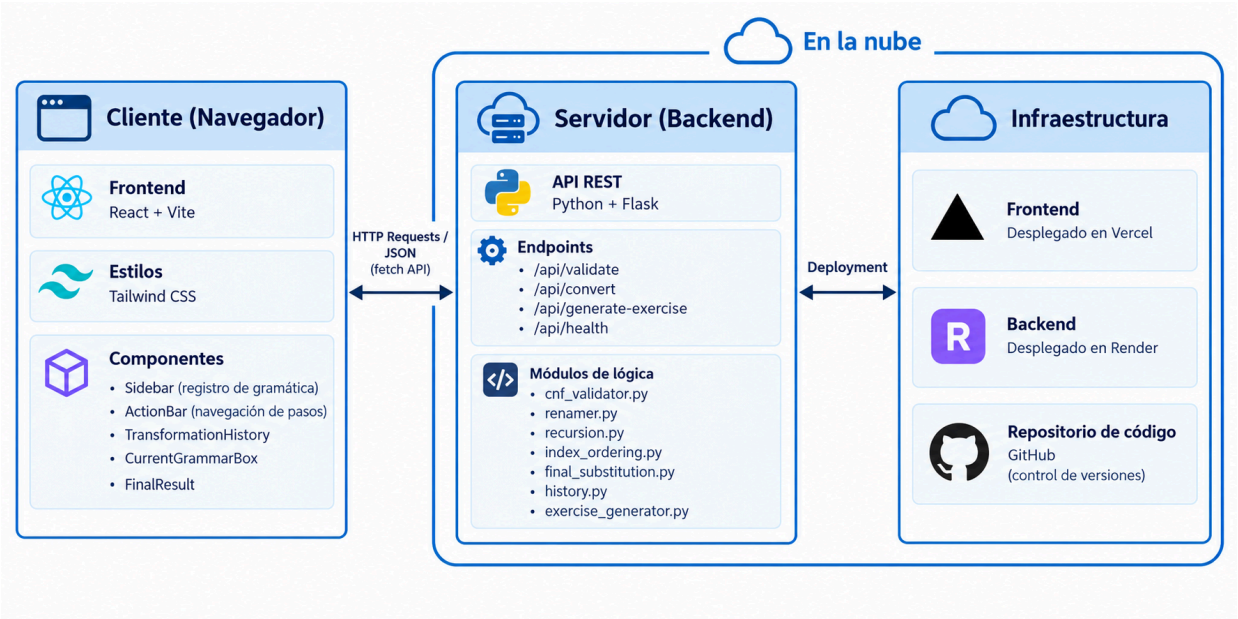
Figura 4. Diagrama de flujo del algoritmo



Nota. Elaborado con IA

DIAGRAMA GENERAL DEL SOFTWARE

Figura 5. Diagrama general del software



Nota. Elaborado con IA

5. ALGORITMOS DE CONVERSIÓN

El algoritmo implementado transforma una gramática que cumple la Forma Normal de Chomsky (FNC) en una gramática equivalente en Forma Normal de Greibach (FNG), es decir, una gramática en la que toda producción tiene la forma A → aα, donde a es un símbolo terminal y α es una cadena (posiblemente vacía) de variables. A continuación se describe cada una de las fases del algoritmo, en el orden exacto en que se ejecutan.

Validación de FNC

Antes de iniciar cualquier transformación, se verifica que la gramática de entrada cumpla estrictamente la Forma Normal de Chomsky. Esta validación recorre todas las producciones y comprueba, para cada una, que tenga una de las siguientes formas permitidas:

- $A \rightarrow BC$ (dos variables)
- $A \rightarrow a$ (un único terminal)
- $S \rightarrow \varepsilon$ (únicamente permitida si S es el símbolo inicial y no aparece en el lado derecho de ninguna producción)

Cualquier producción que no se ajuste a estas formas genera un código de error específico (VAL001 a VAL010), con un mensaje descriptivo que indica cuál es la regla incumplida. Si la gramática no supera esta validación, el proceso se detiene de inmediato y se informa al usuario, sin ejecutar ninguna de las fases posteriores: no tiene sentido aplicar el algoritmo de Greibach sobre una gramática que no parte de una base válida en FNC.

Ordenamiento de variables (renombrado)

Una vez validada la gramática, el primer paso efectivo del algoritmo es imponer un orden total sobre las variables, renombrándolas como X1, X2, …, Xn, de modo que X1 sea siempre el símbolo inicial. Este ordenamiento es indispensable porque el algoritmo de Greibach depende de poder comparar índices entre variables (i frente a j) para decidir qué producciones deben sustituirse.

El renombrado conserva la estructura de la gramática: solo cambian los nombres de las variables, nunca los cuerpos de las producciones ni el lenguaje generado. El resultado de este

paso es un mapa de equivalencia (variable original → nueva variable) que se muestra al usuario para que pueda seguir la correspondencia a lo largo de todo el proceso.

Detección y eliminación de recursividad izquierda indirecta

La recursividad izquierda indirecta ocurre cuando dos o más variables dependen cíclicamente entre sí para producir una cadena que comienza con una de ellas misma (por ejemplo, Xi → Xj… y, transitivamente, Xj → … → Xi…). Esta forma de recursividad no es tan evidente como la recursividad inmediata (Xi → Xiα), por lo que debe detectarse antes de intentar eliminar la recursividad directa.

Para detectarla, se construye un grafo de dependencias entre variables (una arista Xi → Xj existe si alguna producción de Xi comienza con Xj) y se calculan sus componentes fuertemente conexas mediante el algoritmo de Kosaraju. Cada componente con más de un elemento (o con un ciclo propio) representa un conjunto de variables mutuamente recursivas que debe resolverse.

La eliminación se realiza mediante **sustitución simultánea por rondas**: en cada ronda, toda producción de una variable del ciclo que comienza con otra variable del mismo ciclo se sustituye por las producciones actuales de esa variable, expandiendo la cadena un nivel más. Este proceso se repite ronda tras ronda hasta que ya no queden producciones que inicien en una variable del propio ciclo. Para evitar que esta expansión genere producciones duplicadas (y, con ello, un crecimiento combinatorio innecesario), cada producción nueva se compara con las ya generadas mediante una clave (cabeza, cuerpo) antes de agregarse; si ya existe una producción idéntica, no se vuelve a añadir.

Eliminación de recursividad izquierda inmediata

Una vez resuelta la recursividad indirecta, puede seguir habiendo producciones de la forma Xi → Xiα (recursividad inmediata), ya sea porque existían desde el principio o porque aparecieron como resultado del paso anterior. Para cada variable Xi con este tipo de producciones, se aplica la transformación estándar de eliminación de recursividad izquierda:

- Se separan las producciones de Xi en dos grupos: las **no recursivas** (Xi → β, donde β no comienza con Xi) y las **recursivas** (Xi → Xiα).
- Se crea una nueva variable auxiliar Xi.1.
- Las producciones de Xi se reescriben como Xi → β Xi.1 para cada β no recursiva (y Xi → β si se permite la cadena vacía en Xi.1).
- Las producciones de Xi.1 se definen como Xi.1 → α Xi.1 y Xi.1 → α para cada α recursiva.

Esta transformación elimina la recursividad inmediata sin alterar el lenguaje generado por la variable Xi. Al igual que en el paso anterior, se mantienen dos conjuntos de control de duplicados uno para las producciones de Xi y otro para las de Xi.1 de modo que ninguna alternativa se repita.

Eliminación de producciones con índice menor (orden de índices)

Tras los dos pasos anteriores, puede ocurrir que una variable Xi tenga producciones que comiencen con una variable de índice **menor**, Xi → Xjα con j < i. Esto contradice el orden que el algoritmo necesita mantener (toda variable solo debe depender de variables de índice mayor, hasta llegar a producciones que inicien en terminal).

Para corregirlo, se recorren las variables en orden creciente de índice (X2, X3, …, Xn) y, para cada una, se sustituye toda producción $X_i \rightarrow X_j\alpha$ con $j < i$ por las producciones **actuales** de X_j concatenadas con α. Como X_j ya fue procesada en una iteración anterior (por ser de índice menor), sus producciones en ese punto ya cumplen la condición deseada, por lo que la sustitución no reintroduce el problema. Este reemplazo se repite, variable por variable, hasta que ninguna producción de X_i comience con una variable de índice menor a i. Nuevamente, cada producción generada se filtra contra un conjunto de duplicados antes de agregarse.

Sustitución de producciones y sustituciones sucesivas (sustitución final)

Al finalizar el paso anterior, todas las producciones de la gramática tienen una de estas dos formas: comienzan en un terminal, o comienzan en una variable de índice mayor al de su cabeza. El último paso consiste en procesar las variables en orden decreciente, desde Xn hasta X1:

- Si una producción de X_i ya comienza con un terminal, se deja intacta ya que se encuentra en FNG.
- Si una producción de X_i comienza con una variable X_j (necesariamente $j > i$, por el paso anterior), se sustituye X_j por cada una de sus producciones actuales. Cómo X_j se procesó en una iteración anterior (por tener índice mayor), sus producciones a esa altura del proceso ya comienzan en terminal, de modo que la sustitución resultante también comienza en terminal.

Esta sustitución se aplica de forma sucesiva puede requerir varias rondas dentro de la misma variable si la primera sustitución todavía deja alguna producción iniciando en variable hasta que todas las producciones de X_i comiencen en terminal, momento en el cual se avanza a la

variable siguiente (de índice menor). El mismo mecanismo de deduplicación por (cabeza, cuerpo) se aplica aquí, ya que es la fase donde, en gramáticas con alta recursividad mutua entre muchas variables, el riesgo de generar producciones repetidas es mayor.

Generación de nuevas variables

A lo largo de las fases (recursividad inmediata) y, en menor medida, detección y eliminación de recursividad izquierda indirecta hasta la eliminación de producciones con índice menor, el algoritmo introduce nuevas variables auxiliares (Xi.1) cuando es necesario separar la parte recursiva de la no recursiva de una producción. Estas variables se nombran siguiendo la convención <variable base>.1 para mantener trazabilidad con la variable que les dio origen, y se incorporan al conjunto de variables de la gramática en el mismo punto en que se generan, participando desde entonces en todas las fases posteriores (orden de índices y sustitución final) igual que cualquier otra variable.

Conversión a producciones que comienzan con terminal

Este resultado no es una fase independiente, sino la consecuencia directa de haber completado correctamente las fases de deteccion y eliminacion de recursividad a la izquierda hasta la sustitución de producciones y sustituciones sucesivas: al terminar la sustitución final, toda producción de la gramática tiene la forma $Xi \rightarrow a\alpha$, con $a \in \Sigma$ y α una cadena de cero o más variables, que es precisamente la definición de la Forma Normal de Greibach.

Validación final de FNG

Como verificación de cierre, se recorre la gramática resultante y se comprueba, producción por producción, que el primer símbolo del cuerpo sea siempre un terminal (o que el

cuerpo sea ε, únicamente permitido en el símbolo inicial si la gramática original lo contemplaba). Si alguna producción no cumple esta condición, se reporta como un error de validación de FNG, indicando la producción específica que falló; en el flujo normal del algoritmo esto no debería ocurrir, por lo que esta validación funciona como una salvaguarda de consistencia más que como un paso que se espere que falle.

6. IMPLEMENTACIÓN

Lenguaje utilizado

El proyecto se desarrolló con una arquitectura cliente-servidor desacoplada, usando dos lenguajes y ecosistemas distintos para cada capa:

Backend: Python 3, usando el micro-framework Flask para exponer la lógica de conversión como una API REST. Se eligió Python por su legibilidad a la hora de implementar algoritmos sobre estructuras discretas (gramáticas, grafos de dependencias, conjuntos de producciones), y Flask por ser un framework liviano, suficiente para un conjunto reducido de endpoints sin necesidad de la complejidad de un framework más grande.

Frontend: JavaScript (React) sobre Vite como bundler y servidor de desarrollo, con Tailwind CSS para el sistema de estilos. React se usó por su modelo de componentes y estado reactivo, adecuado para representar la revelación progresiva de los pasos del algoritmo sin recargar la página ni duplicar peticiones al backend.

Comunicación: HTTP/JSON mediante fetch, con CORS habilitado en el backend para permitir que el frontend (desplegado en Vercel) consuma la API (desplegada en Render) desde un origen distinto.

Clases o módulos

El backend se organizó en módulos con responsabilidad única, siguiendo de cerca las fases del algoritmo descritas en el capítulo anterior:

Tabla 3

Tabla módulos

MÓDULO	RESPONSABILIDAD
api.py	Define el servidor Flask y los endpoints HTTP (/api/validar, /api/parsear, /api/convertir, /api/generar-ejercicio, /api/salud). Traduce el JSON de entrada a objetos del dominio y las respuestas de vuelta a JSON.
models.py	Define las estructuras de datos del dominio (Gramatica, Producción) y el parseo de la gramática recibida desde el frontend.
validador_fnc.py	Implementa la validación de la Forma Normal de Chomsky (códigos VAL001–VAL010).
renombrador.py	Implementa el paso 1: renombrado de variables a la notación X1…Xn.
recursividad.py	Implementa los pasos 2 y 3: eliminación de recursividad indirecta (mediante componentes fuertemente conexas) y recursividad inmediata (mediante variable auxiliar).
orden_indices.py	Implementa el paso 4: eliminación de producciones Xi → Xjα con j < i.
sustitucion_final.py	Implementa el paso 5: sustitución final hasta Forma Normal de Greibach, y la validación final de FNG.
historial.py	Orquesta los cinco pasos en secuencia, controla el límite global de producciones y serializa cada paso (con su gramática

	resultante y sus eventos) a un formato listo para enviar como JSON.
generador_ejercicios.py	Genera gramáticas FNC aleatorias, usadas por la función de “generar ejercicio” de la aplicación.

Nota. Elaboración propia

El frontend se organizó como un árbol de componentes React, cada uno encargado de una sección visual concreta:

Tabla 4

Tabla componentes

COMPONENTE	RESPONSABILIDAD
App.jsx	Componente raíz. Mantiene el estado global (gramática registrada, resultado del backend, paso máximo visible) y coordina las llamadas a la API.
Encabezado.jsx	Cabecera de la aplicación.
Sidebar.jsx	Formulario de registro de la gramática (variables, terminales, símbolo inicial, producciones), con validación en tiempo real, y panel de “Gramática original”.
BarraAcciones.jsx	Botones de navegación: validar, ejecutar paso a paso, ejecutar proceso completo, nueva gramática.
ValidacionPanel.jsx	Muestra los resultados de la validación de FNC.
GramaticaActualBox.jsx	Muestra la gramática resultante del paso actualmente visible, con su notación de tupla.
HistorialTransformaciones.jsx	Muestra, paso a paso, las producciones generadas, marcando en color las que son nuevas respecto al paso anterior.

ResultadoFinal.jsx	Muestra la gramática final en FNG una vez se completa el proceso.

Nota. Elaboración propia

Métodos

Los métodos (funciones) más relevantes del backend son:

- parsear_lineas_gramatica(variables, terminales, inicial, lineas) *(models.py)*: construye un objeto Gramatica a partir de las líneas de texto recibidas del frontend (formato A -> AB/CD/a).
- validar_fnc(g) *(validador_fnc.py)*: recorre todas las producciones de g y devuelve la lista de errores encontrados respecto a la Forma Normal de Chomsky.
- renombrar_a_greibach(g, orden_manual=None) *(renombrador.py)*: devuelve una nueva gramática con las variables renombradas a X1…Xn, y el evento con el mapa de equivalencia. Acepta un orden manual opcional si el usuario desea fijar el orden de las variables.
- eliminar_recursividad_indirecta(g) *(recursividad.py)*: calcula las componentes fuertemente conexas del grafo de dependencias entre variables y sustituye simultáneamente, por rondas, hasta eliminar los ciclos.
- eliminar_recursividad_inmediata(g) *(recursividad.py)*: para cada variable con producciones Xi → Xiα, separa las producciones recursivas de las no recursivas y crea la variable auxiliar Xi.1.
- eliminar_orden_indices(g) *(orden_indices.py)*: sustituye las producciones Xi → Xjα con j < i por las producciones actuales de Xj, repitiendo hasta que se respete el orden.

- sustitucion_final(g) y validar_fng(g) *(sustitucion_final.py)*: procesan las variables de la última a la primera sustituyendo toda producción que aún inicie en variable, y validan que el resultado cumpla la Forma Normal de Greibach.
- convertir_a_fng(g, orden_manual=None) *(historial.py)*: orquesta los cinco pasos anteriores en orden, verifica el límite de producciones tras cada paso (_verificar_limite) y construye el historial completo con los eventos de cada transformación.
- generar_ejercicio(num_variables, num_terminales, …) *(generador_ejercicios.py)*: genera una gramática FNC aleatoria válida, usada como punto de partida para que el usuario practique.

  En el frontend, las funciones centrales viven en App.jsx:

- registrar(payload): guarda la gramática ingresada en el estado y reinicia el resultado y la validación.
- asegurarResultado(): llama una única vez al backend (/api/convertir) y cachea el resultado completo, evitando peticiones repetidas al navegar entre pasos.
- irAPaso(indice) y ejecutarCompleto(): controlan cuántos pasos del historial ya calculado se revelan en pantalla (pasoMaximoVisible), sin necesidad de recalcular nada en el backend.

Estructuras de datos

Las estructuras de datos centrales del backend son:

- **Produccion**: representa una única alternativa de producción, con los campos cabeza (la variable, por ejemplo "X1") y cuerpo (una tupla de símbolos, por ejemplo ("a", "X2")).

- **Gramatica**: agrupa la lista de variables, la lista de terminales, el símbolo inicial y la lista de producciones (objetos Produccion). Incluye métodos auxiliares como producciones_de(variable), que filtra las producciones cuya cabeza coincide con una variable dada, y texto(), que serializa la gramática a una representación legible.
- **vistos: Set[Tuple[str, tuple]]**: estructura de control usada en recursividad.py, orden_indices.py y sustitucion_final.py para evitar producciones duplicadas. Cada producción se identifica de forma única por el par (cabeza, cuerpo); antes de agregar una producción generada por una sustitución, se comprueba si esa clave ya existe en el conjunto, y solo se agrega si es nueva. Esta es la estructura que resolvió el problema de explosión combinatoria de producciones en gramáticas con muchas variables mutuamente recursivas.
- **Grafo de dependencias entre variables**: usado en eliminar_recursividad_indirecta, representado como un diccionario de adyacencia (variable → conjunto de variables de las que depende directamente), sobre el cual se ejecuta el algoritmo de Kosaraju para obtener las componentes fuertemente conexas.
- **ExplosionDeProducciones**: excepción personalizada, lanzada por historial.py cuando, tras cualquiera de los cinco pasos, el número de producciones supera el límite (MAX_PRODUCCIONES = 6000), para evitar el agotamiento de memoria ante gramáticas patológicas.

En el frontend, el estado se organiza principalmente como:

- gramaticaRegistrada: objeto con la gramática tal como la ingresó el usuario (variables, terminales, inicial, producciones).

- resultado: objeto devuelto por el backend con los cinco pasos completos (pasos: [...]), cada uno con su gramática resultante y sus eventos.
- pasoMaximoVisible: número entero que controla cuántos de los pasos ya calculados se muestran en pantalla, permitiendo un modo paso a paso sin volver a llamar al backend.

Funcionamiento general

El flujo completo de la aplicación es el siguiente:

- El usuario ingresa la gramática en el Sidebar (variables, terminales, símbolo inicial, producciones), con validación en tiempo real sobre el formato de los símbolos.
- Al registrar la gramática, el frontend guarda una copia inmutable ("Gramática original") y limpia cualquier resultado previo.
- El usuario puede validar la gramática (/api/validar), ejecutar el proceso paso a paso (un paso a la vez) o ejecutar el proceso completo de una vez (/api/convertir).
- En cualquiera de los dos modos, el backend calcula **siempre** el pipeline completo de 5 pasos en una sola llamada (convertir_a_fng); lo que cambia es cuántos de esos pasos revela el frontend de inmediato (pasoMaximoVisible), evitando llamadas redundantes a la API al navegar entre pasos ya calculados.
- Cada paso devuelto incluye la gramática resultante, los eventos de la transformación (qué se sustituyó y por qué) y las producciones marcadas como "nuevas" o "normales" respecto al paso anterior, lo que permite al HistorialTransformaciones resaltar visualmente los cambios.

- Si en algún punto el número de producciones supera el límite configurado, el backend aborta la conversión con un código de error específico (explosion), que el frontend muestra como un mensaje claro en lugar de fallar silenciosamente.
- Al completarse el proceso, se muestra la gramática final en Forma Normal de Greibach junto con el resultado de su validación (validar_fng).

7. PRUEBAS

Para validar el correcto funcionamiento del algoritmo se diseñaron y ejecutaron casos de prueba de complejidad creciente, ejecutados directamente contra los módulos del backend. A continuación se documenta cada caso siguiendo la estructura exigida: gramática inicial, validación FNC, transformaciones, gramáticas intermedias, gramática final y validación FNG.

Caso 1 Gramática simple (sin recursividad)

Gramática inicial:

S -> AB

A -> a

B -> b

Validación FNC: válida (cumple la forma VntVnt en S y Vt en A y B).

Transformaciones: al no existir ninguna producción que inicie en variable de forma recursiva ni fuera de orden, los pasos 2, 3 y 4 (recursividad indirecta, recursividad inmediata y orden de índices) no modifican la gramática; solo el paso 1 (renombrado) y el paso 5 (sustitución final) producen cambios.

Gramáticas intermedias:

Paso 1 (renombrado): X1 -> X2X3 | X2 -> a | X3 -> b

Pasos 2, 3 y 4 (sin cambio): X1 -> X2X3 | X2 -> a | X3 -> b

Gramática final (FNG):

X1 -> aX3

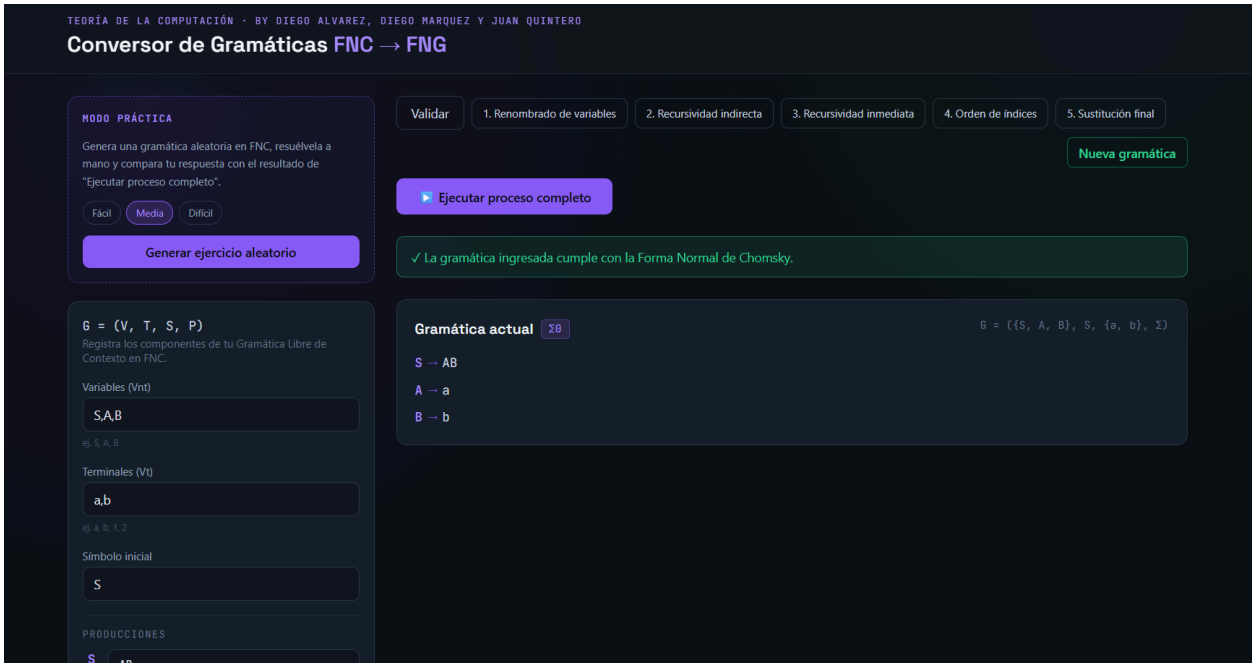
X2 -> a

X3 -> b

Validación FNG: válida las tres producciones inician en terminal.

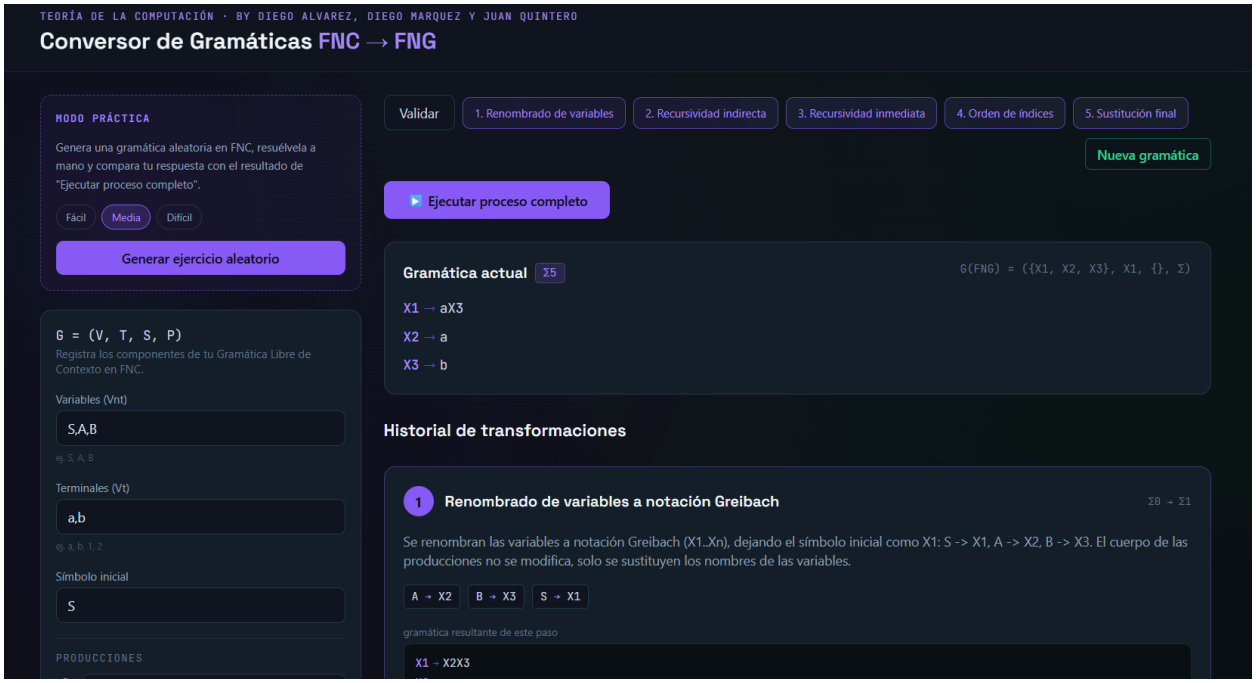
Este caso confirma que, ante una gramática ya cercana a la forma final, el algoritmo no introduce transformaciones innecesarias y produce el resultado mínimo esperado.

Figura 6. Validación FNC



Nota. Capture tomado del conversorfng

Figura 7. Ejecución proceso completo



Nota. Capture tomado del conversorfng

Caso 2 Gramática media (recursividad inmediata e indirecta combinadas)

Gramática inicial:

S -> AB | b

A -> SA | a

B -> b

Validación FNC: válida.

Transformaciones: tras el renombrado (S→X1, A→X2, B→X3), X2 -> X1X2 introduce una dependencia de X1 hacia X2 y de vuelta a X1, por lo que el paso 2 detecta un ciclo de

recursividad indirecta entre X1 y X2 y sustituye simultáneamente hasta romperlo. El resultado de ese paso deja a X1 con una producción X1 -> X1X2X3, es decir, recursividad inmediata, que el paso 3 resuelve creando la variable auxiliar X1.1. Lo mismo ocurre con X2, que genera X2.1. El paso 4 no encuentra producciones fuera de orden, y el paso 5 sustituye las referencias a X2 y X3 dentro de X1.1 hasta que todo comienza en terminal.

Gramáticas intermedias:

Paso 1: X1 -> X2X3 | b X2 -> X1X2 | a X3 -> b

Paso 2: X1 -> X1X2X3 | aX3 | b X2 -> X2X3X2 | bX2 | a X3 -> b

Paso 3: X1 -> aX3 | b | aX3X1.1 | bX1.1 X1.1 -> X2X3 | X2X3X1.1

X2 -> bX2 | a | bX2X2.1 | aX2.1 X2.1 -> X3X2 | X3X2X2.1 X3 -> b

Paso 4: (sin cambios adicionales)

Gramática final (FNG):

X1 -> aX3 | b | aX3X1.1 | bX1.1

X1.1 -> bX2X3 | aX3 | bX2X2.1X3 | aX2.1X3 | bX2X3X1.1 | aX3X1.1 | bX2X2.1X3X1.1 | aX2.1X3X1.1

X2 -> bX2 | a | bX2X2.1 | aX2.1

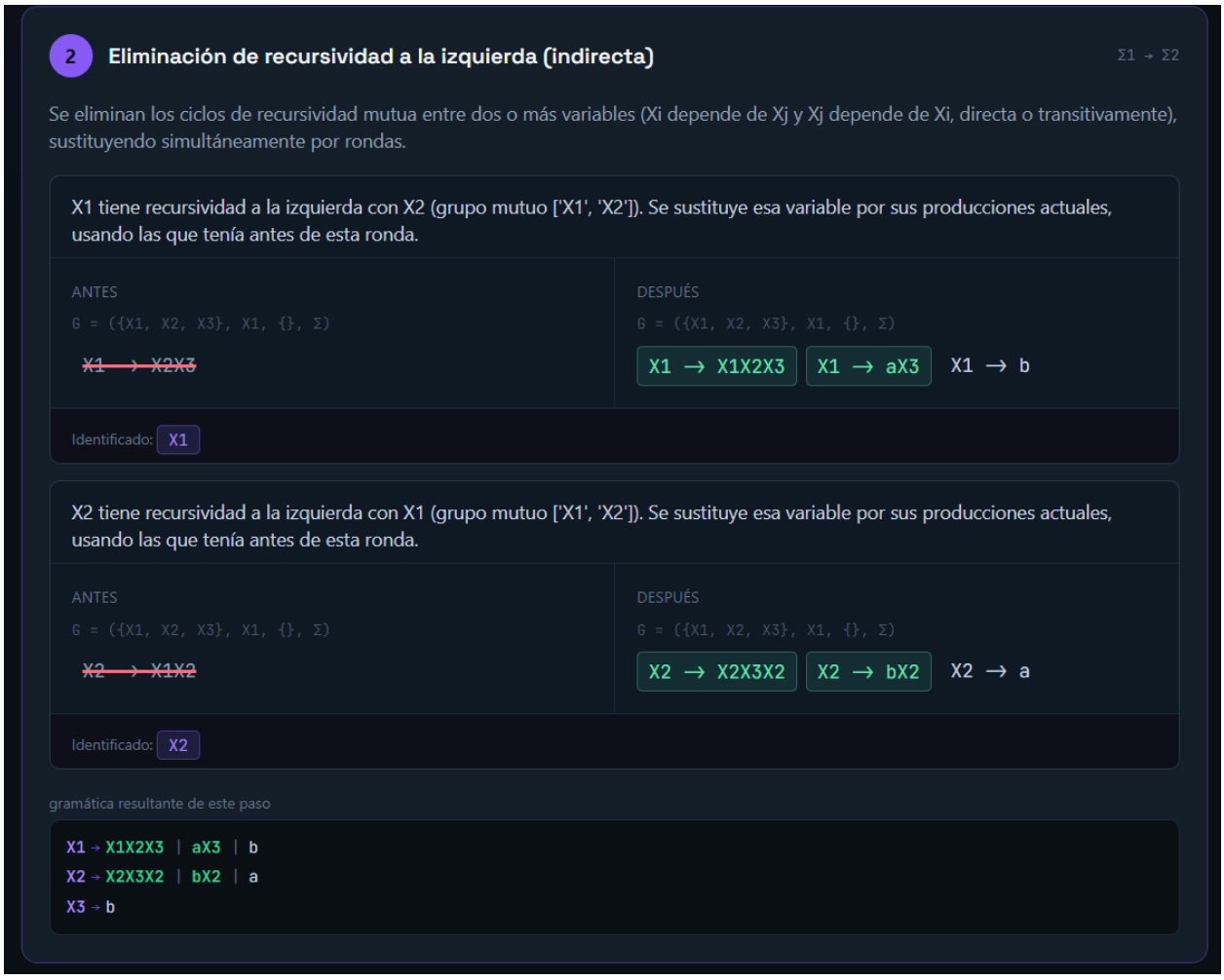
X2.1 -> bX2 | bX2X2.1

X3 -> b

Validación FNG: válida las 19 alternativas de las 5 variables (3 originales + 2 auxiliares) inician en terminal.

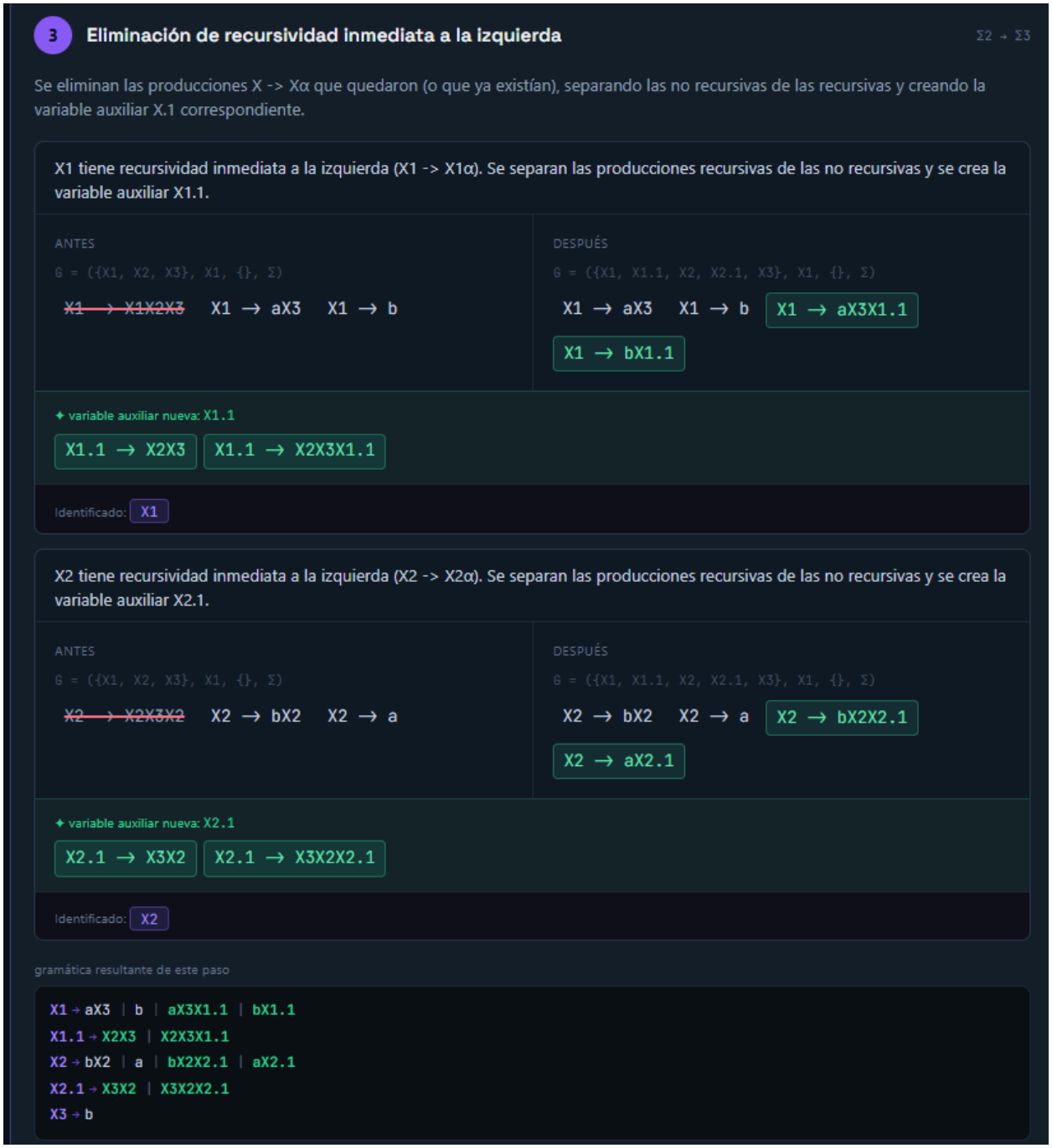
Este caso confirma que la combinación de los pasos 2 (recursividad indirecta) y 3 (recursividad inmediata) funciona correctamente en cascada, y que la variable auxiliar X1.1 queda bien resuelta aun cuando su cuerpo depende de otra variable (X2) que también generó su propia auxiliar (X2.1).

Figura 8. Recursividad inmediata a la izquierda



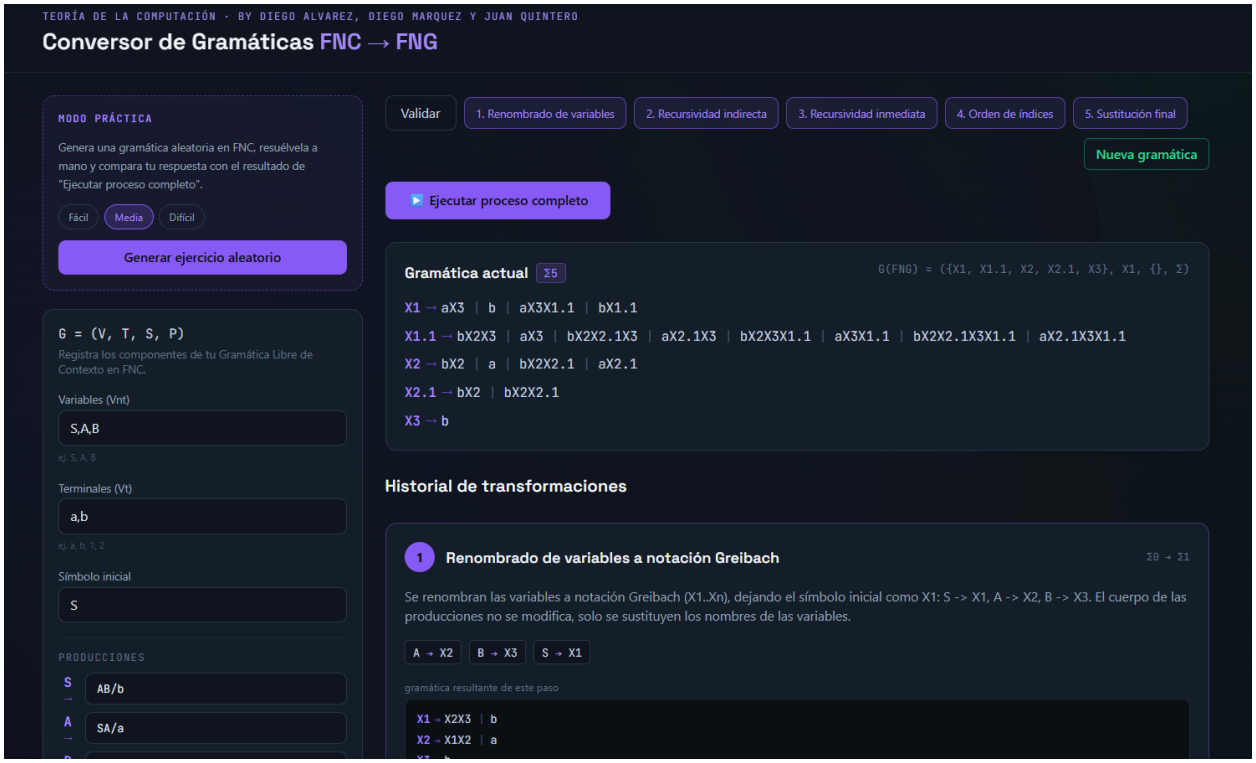
Nota. Capture tomado del conversorfng

Figura 9. Recursividad inmediata a la izquierda



Nota. Capture tomado del conversorfng

Figura 10. Resultado final



Nota. Capture tomado del conversorfng

Caso 3 Gramática compleja (4 variables con recursividad mutua densa)

Gramática inicial:

S -> AB | BC | a

A -> SB | CS | b

B -> SC | c

C -> AB | a

Validación FNC: válida.

Transformaciones: las cuatro variables dependen unas de otras de forma cíclica (S↔A↔B↔C↔S), por lo que el paso 2 debe resolver un único componente fuertemente conexo de tamaño 4. Esto produce un crecimiento notable y esperado en el número de alternativas a medida que avanza el pipeline:

Tabla 5

Pasos FNG

PASO	ALTERNATIVA TOTALES
Renombrado	10
Recursividad indirecta	30
Recursividad inmediata	60
Orden de índices	96
Sustitución final	240

Nota. Elaboración propia

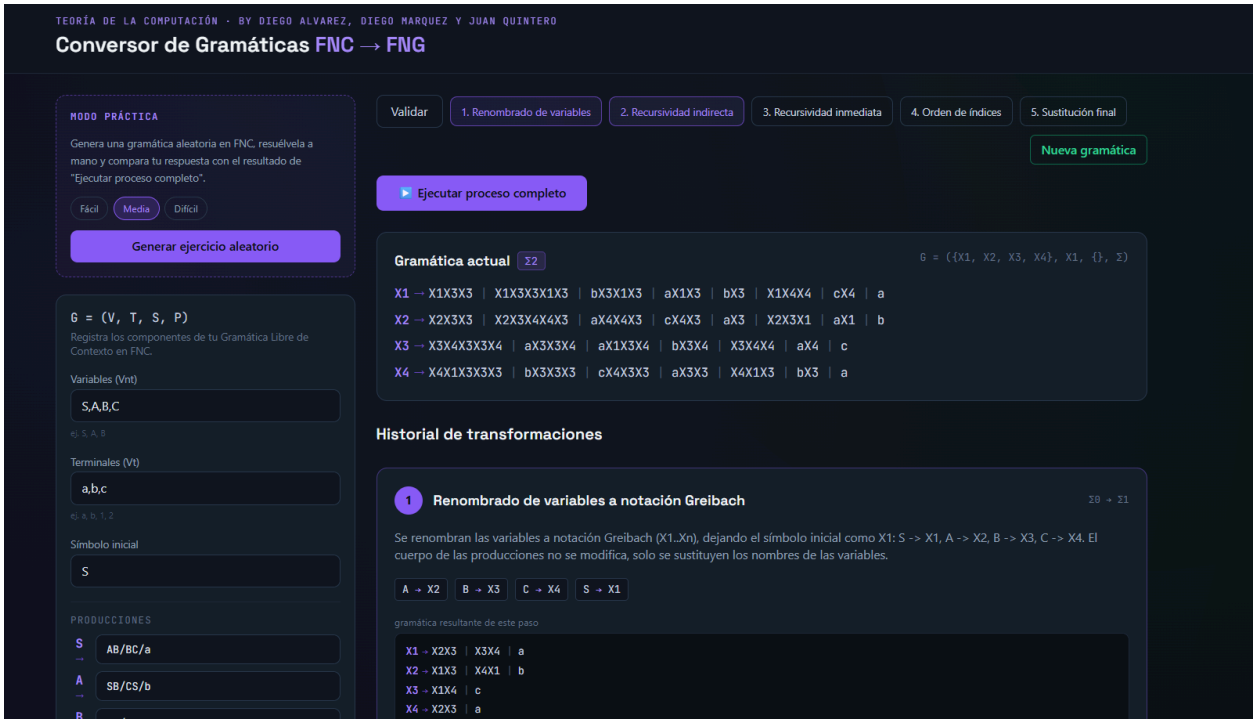
Cada variable original generó su propia variable auxiliar (X1.1, X2.1, X3.1, X4.1), y el crecimiento de 10 a 240 alternativas ilustra, con un caso real, el fenómeno de expansión combinatoria inherente al algoritmo de Greibach cuando hay recursividad mutua entre varias variables: cada sustitución de una variable por sus producciones multiplica el número de alternativas de la variable que la contiene. Gracias a la deduplicación por (cabeza, cuerpo) aplicada en cada ronda, ninguna de esas 240 alternativas es una repetición exacta de otra; el crecimiento observado es, por tanto, genuino y no producto de redundancia.

Gramática final (FNG): las 8 variables resultantes (4 originales + 4 auxiliares) quedan con todas sus producciones iniciando en terminal (a, b o c), con cuerpos que alcanzan hasta 9 símbolos de longitud en los casos más expandidos de X1.1 y X4.1.

Validación FNG: válida.

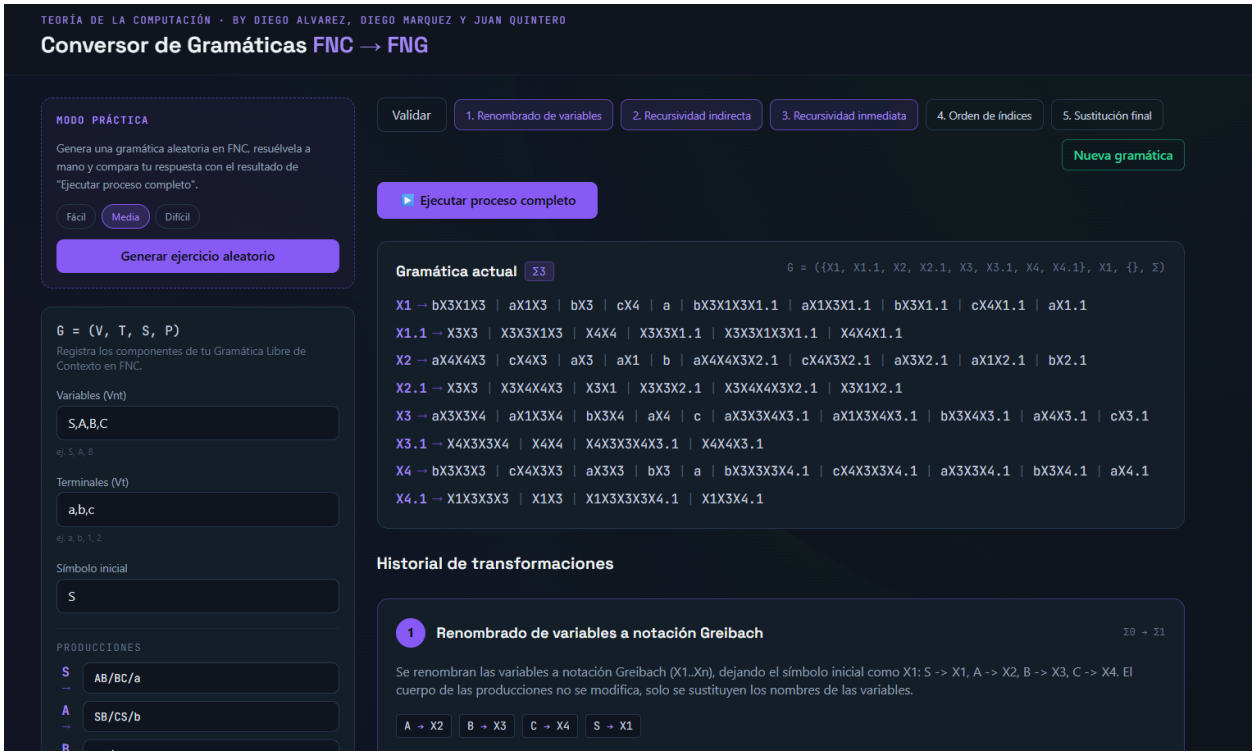
Este caso demuestra que el algoritmo escala correctamente ante gramáticas densamente recursivas de tamaño moderado, y sirve de referencia del comportamiento esperado antes de alcanzar el caso límite del siguiente apartado.

Figura 11. Recursividad a la izquierda



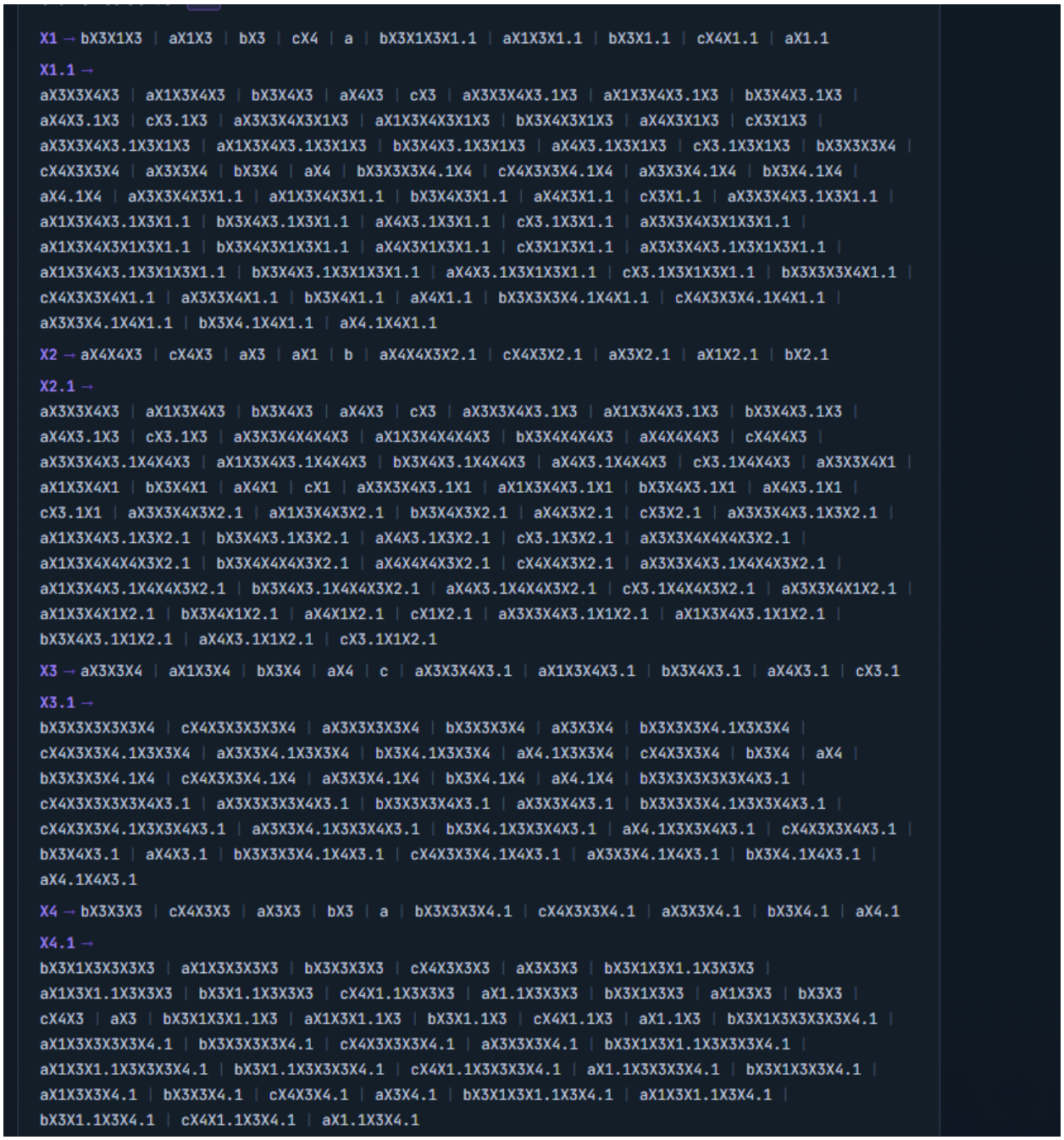
Nota. Capture tomado del conversorfng

Figura 12. Recursividad inmediata a la izquierda



Nota. Capture tomado del conversorfng

Figura 13. Sustitución final



Nota. Capture tomado del conversorfng

Caso 4 Caso límite: control de explosión de producciones

Gramática inicial (8 variables, cada una dependiendo de las dos siguientes en un ciclo cerrado S→A→B→C→D→E→F→G→S):

S -> AB | BC | a

A -> BC | CD | b

B -> CD | DE | c

C -> DE | EF | a

D -> EF | FG | b

E -> FG | GS | c

F -> GS | SA | a

G -> SA | AB | b

Validación FNC: válida.

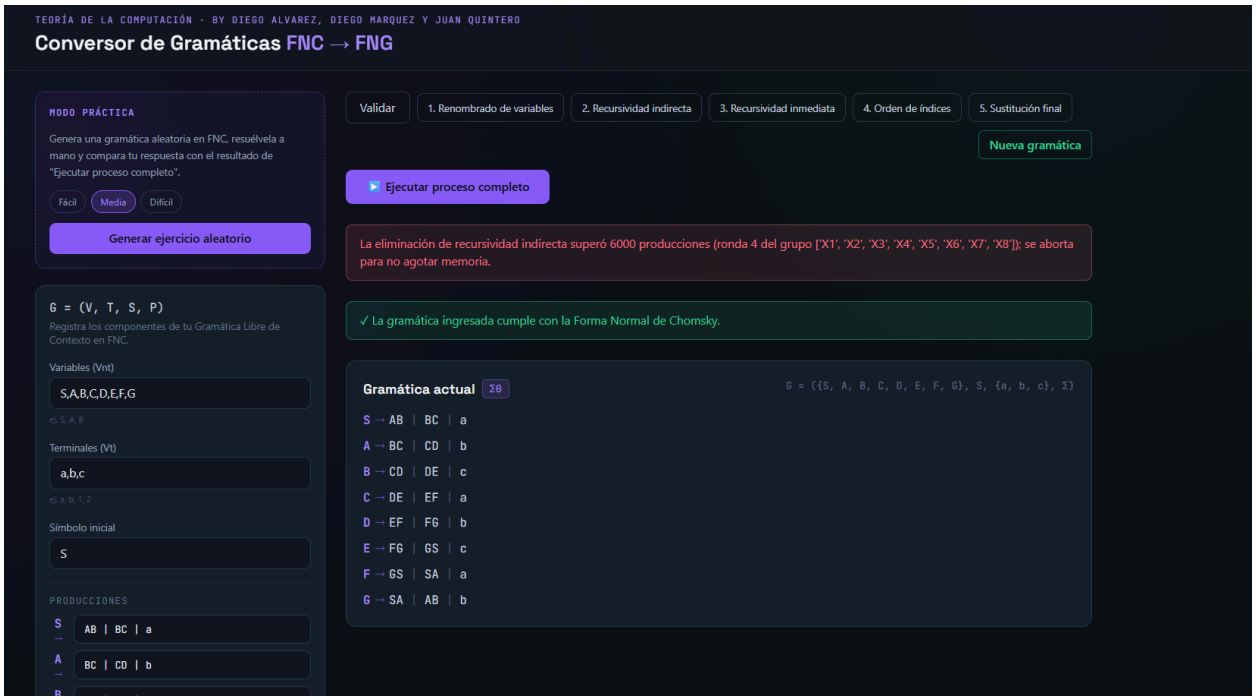
Resultado: al intentar resolver el componente fuertemente conexo formado por las 8 variables (un único ciclo que las conecta a todas), el número de producciones generadas durante las rondas de sustitución simultánea del paso 2 supera el límite configurado (MAX_PRODUCCIONES = 6000) en la cuarta ronda. El sistema interrumpe el proceso de forma controlada y devuelve el siguiente mensaje:

"La eliminación de recursividad indirecta superó 6000 producciones(ronda 4 del grupo ['X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8']);se aborta para no agotar memoria."

El endpoint /api/convertir responde con código HTTP 422 y error_tipo: "explosion", que el frontend muestra como un mensaje de error legible en lugar de bloquearse o agotar memoria del servidor.

Este caso no representa un fallo del algoritmo, sino una verificación de que el mecanismo de seguridad funciona como se diseñó: cuando una gramática combina un número alto de variables con recursividad mutua total entre todas ellas (el peor caso posible para este algoritmo), el crecimiento de producciones es matemáticamente inherente al método de Greibach y no puede evitarse sin cambiar el lenguaje generado. El límite de 6000 producciones existe precisamente para detener ese crecimiento de forma ordenada antes de que comprometa el rendimiento del servidor, en vez de dejar que el proceso corra indefinidamente o agote la memoria disponible.

Figura 14. supera el número de producciones máximo



Nota. Capture tomado del conversorfng

Caso 5 Validación de FNC casos inválidos

Caso A Producción unitaria

S -> A

A -> AB

B -> b

“VAL003: La producción 'S -> A' es una producción unitaria (no terminal -> no terminal). En FNC no se permiten producciones unitarias; deben haberse eliminado en la fase de normalización previa.”

Caso B Cuerpo con más de 2 símbolos

S -> ABA

A -> a

B -> b

“VAL008: La producción 'S -> ABA' tiene 3 símbolos en el cuerpo. En FNC el cuerpo no puede superar 2 símbolos”

Caso C Producción repetida

S -> AB | AB

A -> a

B -> b

“VAL009: La variable 'S' tiene la producción 'S -> AB' repetida 2 veces. Elimina las repeticiones, cada alternativa debe aparecer una sola vez.”

Caso D Variable declarada sin producciones

S -> AB

A -> a

B -> b

(C declarada pero sin ninguna línea de producción)

“VAL001: La variable 'C' está declarada pero no tiene ninguna producción. VAL010: La variable 'C' es inalcanzable desde el símbolo inicial 'S'...”

(este caso dispara dos errores a la vez: una variable sin producciones es, por definición, también inalcanzable salvo que sea el propio símbolo inicial)

Caso E Variable inalcanzable

S -> AB

A -> a

B -> b

C -> c

“VAL010: La variable 'C' es inalcanzable desde el símbolo inicial 'S': no aparece en el cuerpo de ninguna producción derivable desde 'S'. Elimínala o conéctala a la gramática desde alguna producción alcanzable.”

Caso F Múltiples errores combinados (demuestra que el validador no se detiene en el primer error, sino que acumula todos)

S -> A

A -> ABA | ABA

B -> b

C -> c

“VAL003: 'S -> A' es producción unitaria.

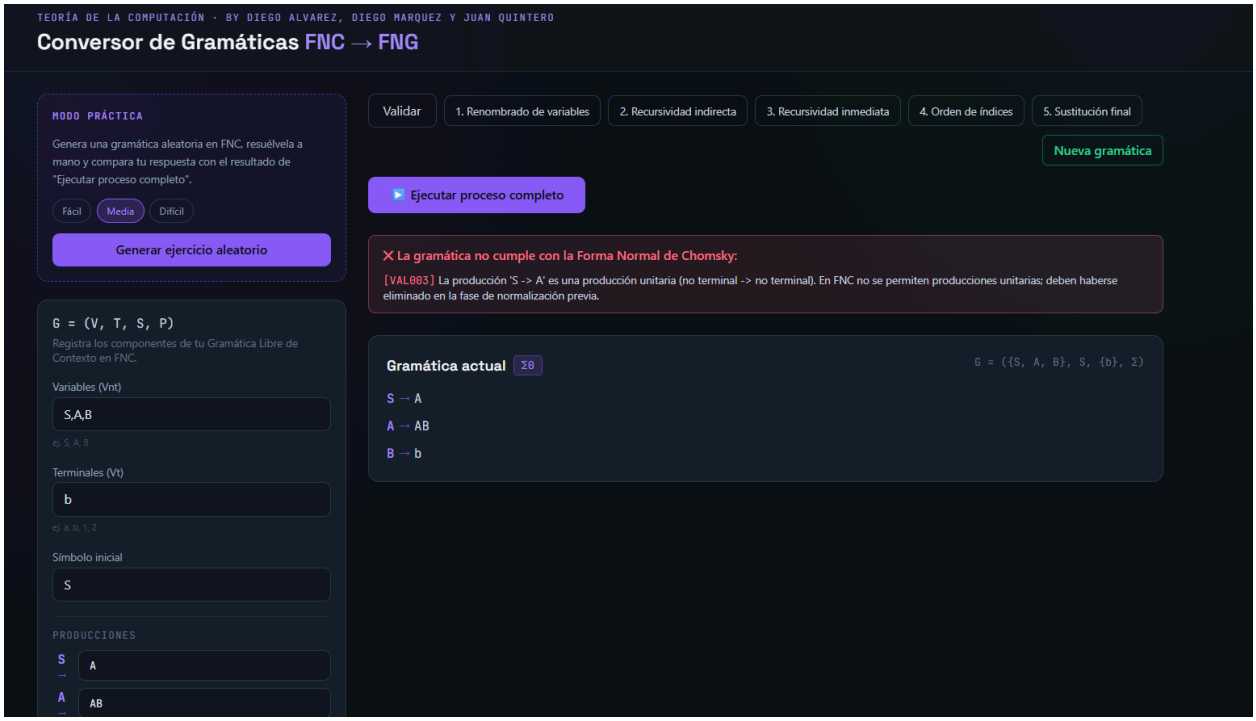
VAL008: 'A -> ABA' tiene 3 símbolos en el cuerpo. (reportado 2 veces, una por alternativa)

VAL009: 'A -> ABA' está repetida 2 veces.

VAL010: 'C' es inalcanzable desde 'S'.”

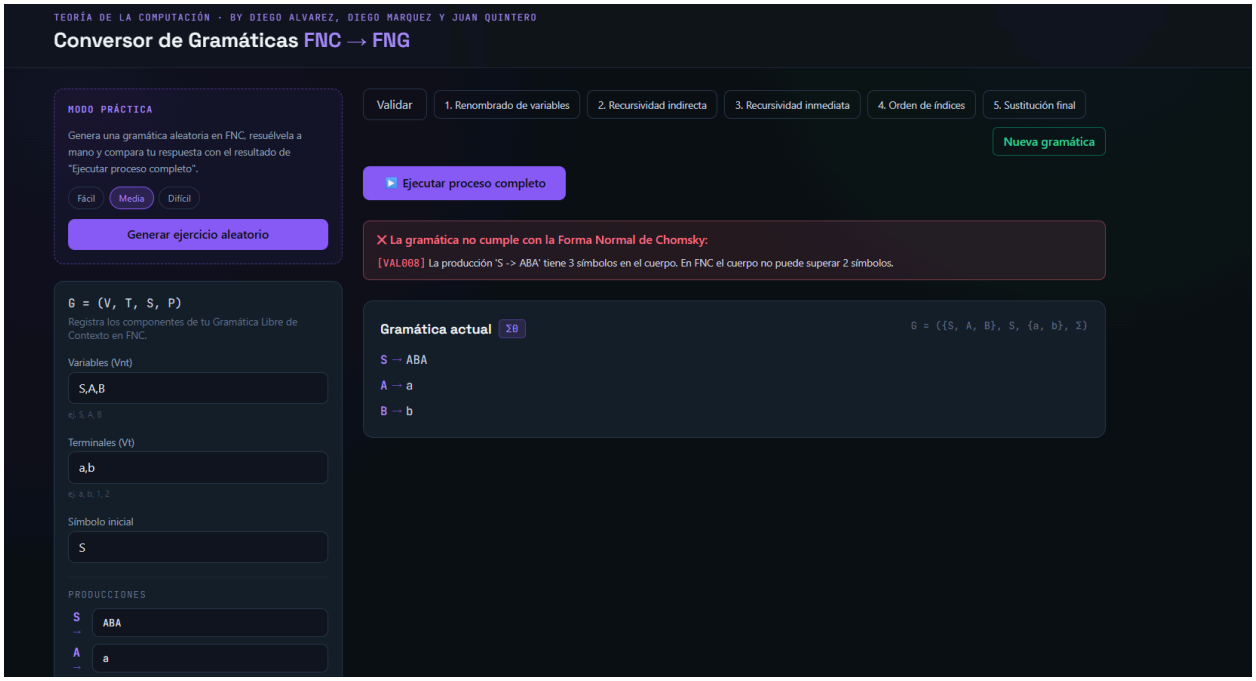
Este caso confirma que la validación es exhaustiva: el usuario ve de una sola vez todos los problemas de su gramática, en vez de corregirlos uno por uno en sucesivas re-validaciones.

Figura 15. Caso A



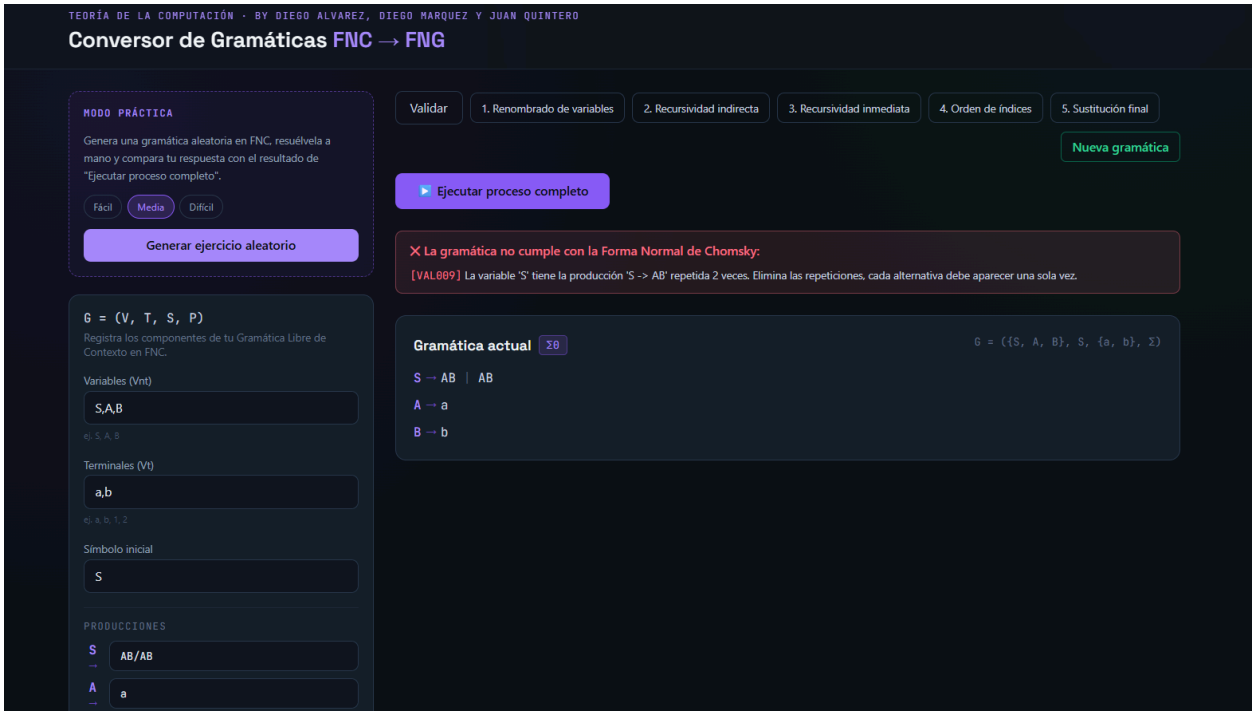
Nota. Capture tomado del conversorfng

Figura 16. Caso B



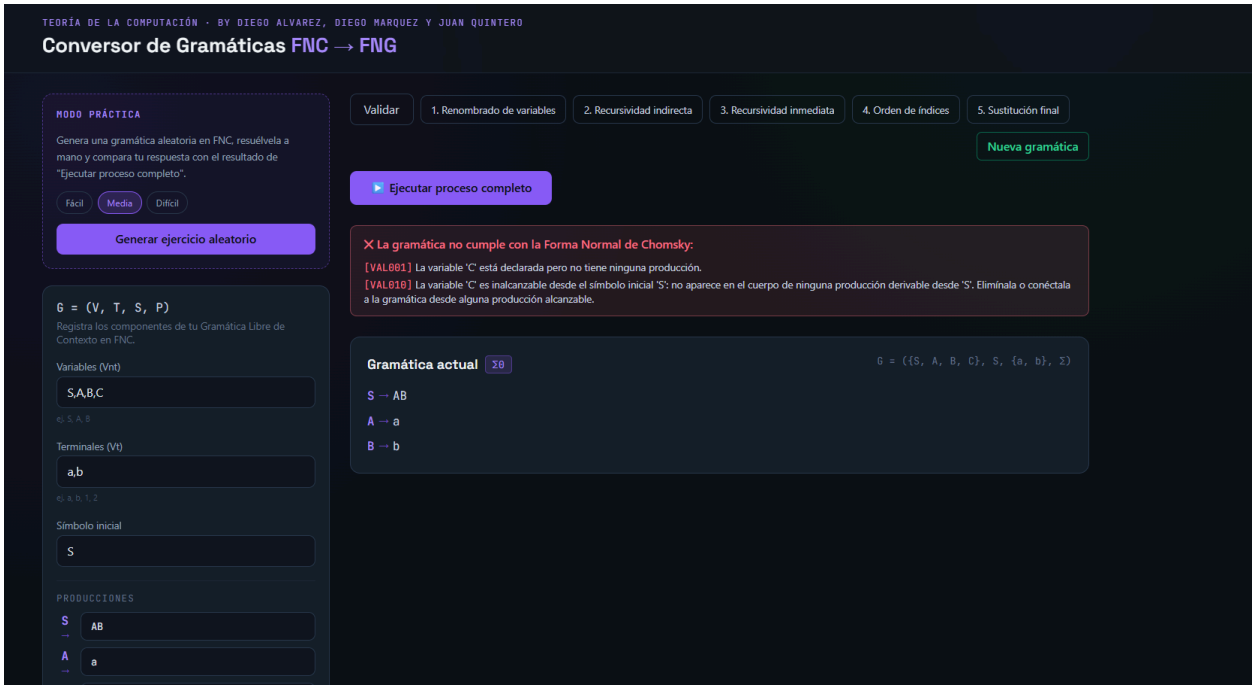
Nota. Capture tomado del conversorfng

Figura 17. Caso C



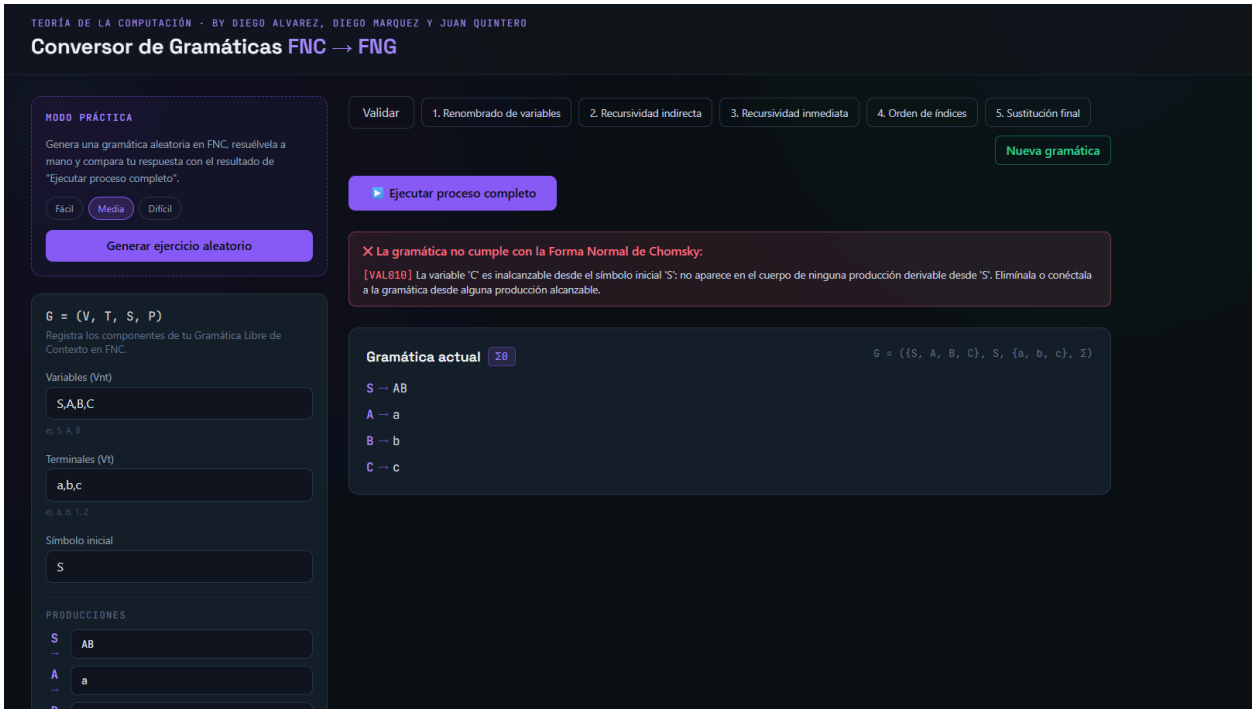
Nota. Capture tomado del conversorfng

Figura 18. Caso D



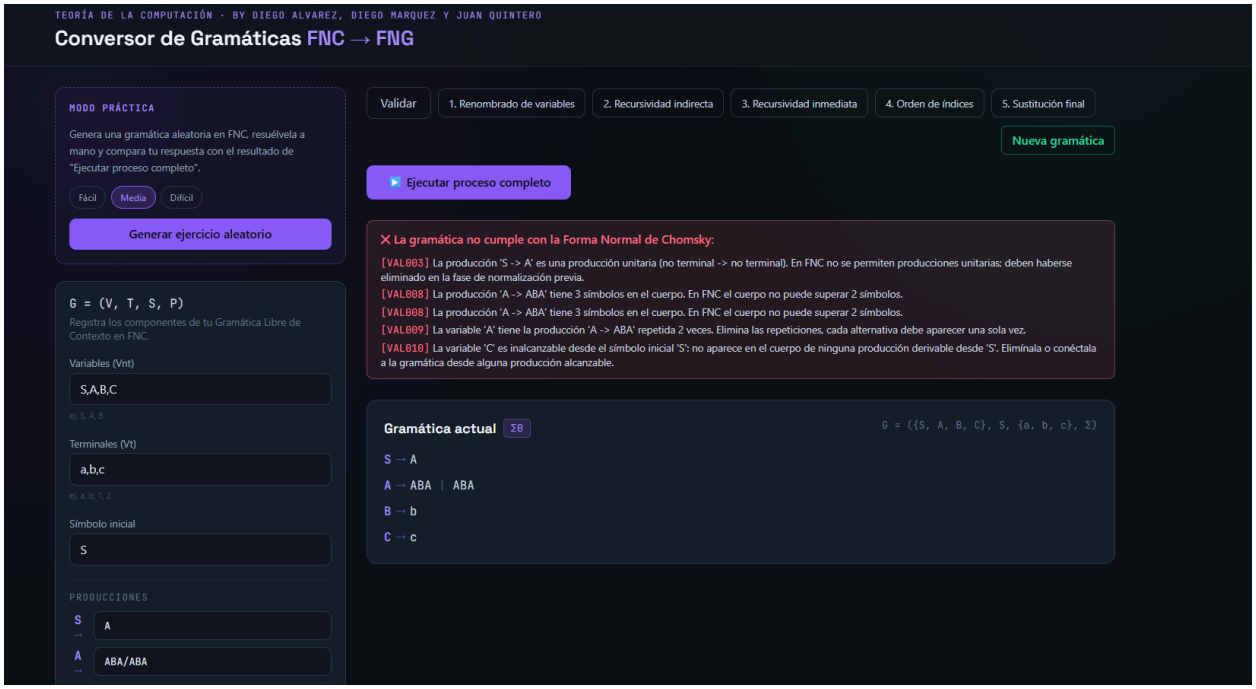
Nota. Capture tomado del conversorfng

Figura 19. Caso E



Nota. Capture tomado del conversorfng

Figura 20. Caso F



Nota. Capture tomado del conversorfng

Caso 6 Validación en tiempo real del formulario de registro

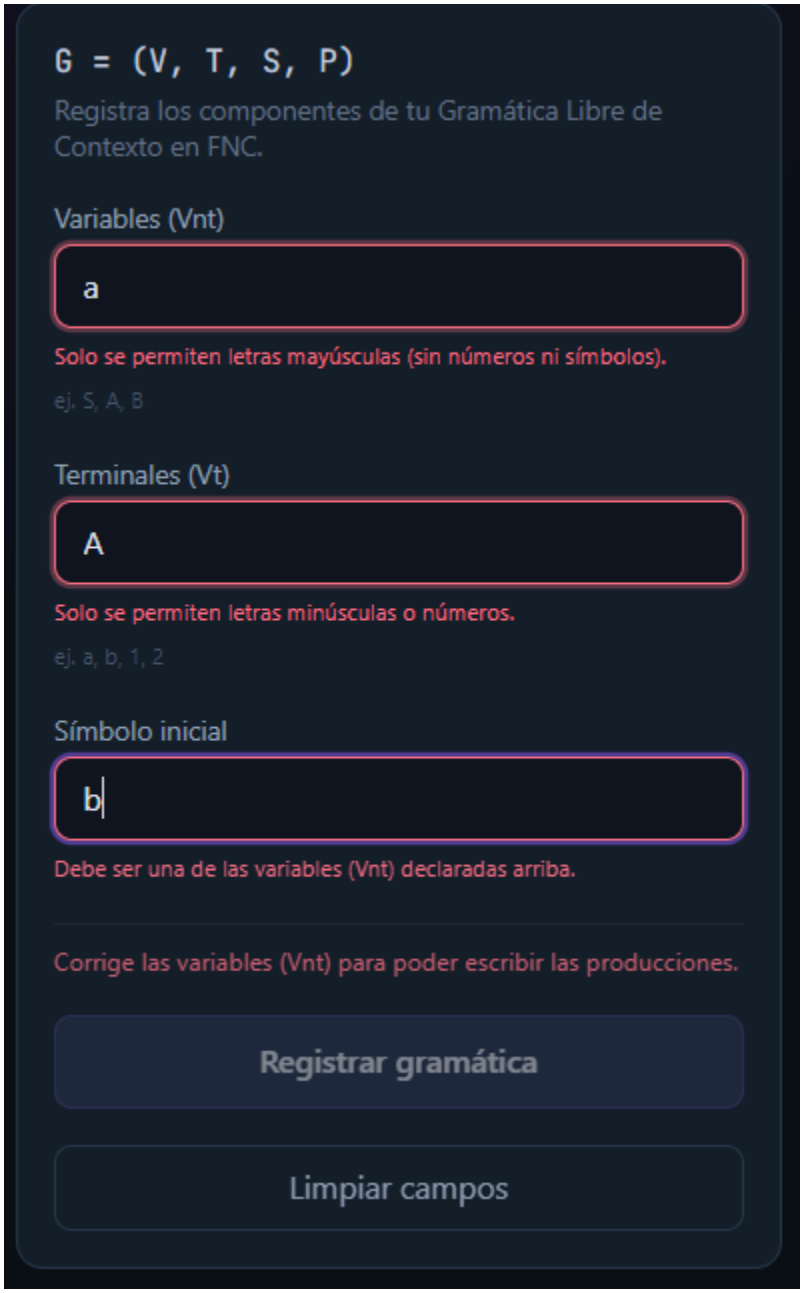
Independientemente de la validación de FNC (que ocurre en el backend al enviar la gramática), el Sidebar aplica una validación de formato en tiempo real sobre lo que el usuario escribe, antes incluso de registrar la gramática:

- **No terminales (Vnt):** deben cumplir la expresión regular ^[A-Z]+$ (una o más letras mayúsculas). Si el usuario escribe un símbolo que no cumple esto (por ejemplo s1 o A-), el campo se marca con **borde rojo** y aparece un mensaje de error debajo, indicando qué símbolo es inválido.

- **Terminales (Vt):** deben cumplir ^[a-z0-9]+$ (letras minúsculas y/o dígitos). Un símbolo como A o # en este campo también activa el borde rojo.
- **Símbolo inicial:** además de cumplir el patrón de Vnt, debe pertenecer obligatoriamente al conjunto de variables (Vnt) ya declaradas. Si el usuario escribe un símbolo inicial que no está en la lista de Vnt (por ejemplo, declaró S, A, B pero puso C como inicial), también se marca en rojo.
- **Bloqueo progresivo:** mientras el conjunto de Vnt no sea válido, la sección "Producciones" del formulario ni siquiera se muestra — no tiene sentido dejar que el usuario escriba producciones si todavía no se sabe cuáles son las variables válidas de esa gramática.

Esta validación es deliberadamente más estricta y anterior a la de FNC: evita que el usuario llegue a enviar al backend una gramática con símbolos mal formados (minúsculas donde van variables, mayúsculas donde van terminales, etc.), que son errores de digitación más que errores conceptuales de la gramática, y los resuelve con retroalimentación inmediata en vez de un viaje de ida y vuelta al servidor.

Figura 21. Validación en tiempo real



Nota. Capture tomado del conversorfng

8. RESULTADOS

Cumplimiento de los requerimientos

El desarrollo del conversor cubrió la totalidad de los requerimientos funcionales (RF01–RF17) y no funcionales (RNF01–RNF12) definidos en el capítulo de Requerimientos. En particular:

- El sistema valida correctamente la Forma Normal de Chomsky de la gramática de entrada antes de iniciar cualquier conversión, rechazando con mensajes específicos (códigos VAL001 a VAL010) cualquier gramática que no cumpla esta precondición verificado en el caso 5.
- El algoritmo de conversión a Forma Normal de Greibach se ejecuta correctamente en los cinco pasos descritos (renombrado, eliminación de recursividad indirecta, eliminación de recursividad inmediata, orden de índices, sustitución final), produciendo en todos los casos de prueba exitosos una gramática final cuya validación de FNG (validar_fng) resultó positiva verificado en los casos 1 a 3.
- Los dos modos de funcionamiento exigidos (paso a paso y automático) operan sobre un mismo resultado calculado una única vez en el backend, revelando progresivamente los pasos en el frontend sin generar peticiones adicionales innecesarias.
- El sistema protege su propio rendimiento ante gramáticas patológicas (muchas variables con recursividad mutua total), interrumpiendo el proceso de forma controlada en lugar de agotar memoria o colgar el servidor verificado en el caso 4.

- La validación de entrada en tiempo real (formato de variables, terminales y símbolo inicial) evita que lleguen al backend gramáticas mal formadas por errores de digitación, mejorando la experiencia de uso verificado en el apartadode validacion en tiempo real.

La única desviación identificada respecto al alcance formal es la ausencia de una validación explícita de variables **inútiles** (no generadoras), señalada en el caso 6 el sistema solo valida alcanzabilidad (VAL010), no generación, lo cual se documenta como una limitación conocida.

Resultados de las pruebas

La batería de pruebas ejecutada (capítulo 7) confirma el comportamiento esperado del algoritmo en todo el espectro de complejidad de entrada:

Tabla 6

Resultados de las pruebas

DIMENSIÓN EVALUADA	RESULTADO
Gramáticas sin recursividad	Conversión directa, sin pasos intermedios con cambios (caso 1)
Recursividad inmediata e indirecta combinadas	Resuelta correctamente en cascada entre los pasos 2 y 3 (caso 2)
Recursividad mutua densa entre varias variables	Conversión exitosa con crecimiento controlado y sin duplicados (caso 3: 10 → 240 alternativas)
Caso límite (8 variables, ciclo total)	Abortado de forma segura por el mecanismo de control de explosión (caso 4)
Gramáticas fuera de FNC (unitarias, cuerpos largos, repetidas, inalcanzables)	Detectadas y reportadas correctamente, con posibilidad de múltiples errores simultáneos

	(caso 5)
Entrada mal formada en el formulario	Bloqueada antes de llegar al backend, con retroalimentación visual inmediata (caso 6)

Nota. Elaboración propia

Un hallazgo relevante del proceso de pruebas fue la corrección del error de explosión de producciones descrito en el capítulo de Implementación: antes de incorporar la deduplicación por (cabeza, cuerpo) en las fases de recursividad, orden de índices y sustitución final, gramáticas con 8 variables mutuamente recursivas superaban el límite de producciones incluso en escenarios donde una gramática equivalente, sin producciones repetidas, no debería haberlo hecho. Tras la corrección, el límite solo se alcanza en el caso genuinamente peor (recursividad mutua total entre todas las variables, caso 4), confirmando que el crecimiento observado ahora corresponde a producciones distintas entre sí y no a redundancia evitable.

Desempeño observado

En las pruebas realizadas, la conversión completa (los cinco pasos) se ejecuta en una única llamada al backend, con tiempos de respuesta del orden de milisegundos para gramáticas pequeñas y medianas (casos 1 y 2), y por debajo de un segundo para la gramática compleja de 240 alternativas finales (caso 3), incluso considerando la latencia de red entre el frontend desplegado en Vercel y el backend desplegado en Render. El límite de 6000 producciones (MAX_PRODUCCIONES) resultó suficiente para cubrir gramáticas de uso académico típico (hasta 6-7 variables con recursividad moderada) sin comprometer el tiempo de respuesta ni la estabilidad del servicio.

Estado del despliegue

El sistema se encuentra desplegado y operativo en un entorno de producción accesible públicamente:

- **Frontend:** desplegado en Vercel, sirviendo la aplicación React compilada de forma estática.
- **Backend:** desplegado en Render, exponiendo la API REST en Flask.
- **Comunicación:** el frontend consume la API mediante peticiones HTTP/JSON, configuradas a través de la variable de entorno VITE_API_URL.
- **Control de versiones:** el código fuente completo (frontend y backend) se mantiene versionado en un repositorio de GitHub.

Ambos componentes fueron verificados en conjunto, confirmando que el flujo completo (registro de gramática → validación → conversión → visualización de resultados) funciona de extremo a extremo en el entorno real de despliegue, y no únicamente en el entorno local de desarrollo.

9. CONCLUSIONES

El desarrollo del conversor de gramáticas de la Forma Normal de Chomsky a la Forma Normal de Greibach permitió cumplir con el objetivo general planteado: se implementó una herramienta funcional que automatiza un procedimiento algorítmico que, resuelto manualmente, es propenso a errores incluso para gramáticas de tamaño moderado, especialmente cuando existe recursividad entre múltiples variables. El sistema no solo ejecuta la conversión, sino que la hace visible paso a paso, lo que convierte a la aplicación en una herramienta tanto de solución como de aprendizaje del algoritmo mismo.

En cuanto a los objetivos específicos, todos se alcanzaron: se implementó la validación de la Forma Normal de Chomsky como precondición obligatoria, se desarrollaron los cinco pasos del algoritmo de conversión de forma modular y verificable de manera independiente, se construyó una interfaz que distingue entre modo paso a paso y modo automático sin duplicar el trabajo de cómputo, y se desplegó la aplicación en un entorno accesible públicamente (Vercel para el frontend, Render para el backend), cumpliendo con el requisito de que el producto final fuera una herramienta utilizable y no solo un ejercicio académico de escritorio.

El proceso de pruebas documentado en el capítulo 7 fue particularmente valioso porque permitió detectar y corregir, antes de la entrega final, un problema real de explosión combinatoria de producciones en gramáticas con alta recursividad mutua entre varias variables. Este hallazgo llevó a un rediseño puntual pero importante: la incorporación de un mecanismo de deduplicación de producciones por la clave (cabeza, cuerpo) en las tres fases donde ocurre sustitución (recursividad, orden de índices y sustitución final). La corrección no alteró en ningún momento la lógica ni el orden de sustitución del algoritmo se verificó explícitamente que el procedimiento matemático permaneciera intacto, y el resultado fue un sistema capaz de manejar gramáticas considerablemente más complejas sin sacrificar la corrección del resultado ni la estabilidad del servidor.

Un aprendizaje central del proyecto fue comprobar, de forma práctica y no solo teórica, que la conversión a Forma Normal de Greibach tiene un costo combinatorio inherente cuando existe recursividad mutua entre muchas variables: el crecimiento de 10 a 240 alternativas observado en el caso de prueba de 4 variables (caso 3), y la explosión controlada en el caso de 8 variables con ciclo total (caso 4), no son fallas del algoritmo sino una propiedad matemática del método mismo. Entender esta diferencia entre un error de implementación y un límite estructural

del algoritmo fue clave para decidir correctamente dónde intervenir (deduplicación de producciones) y dónde simplemente aceptar y comunicar el límite (el control de explosión como salvaguarda, no como corrección de fondo).

El proyecto también deja claro el alcance exacto de la validación de símbolos del sistema: si una producción referencia un símbolo que no fue declarado ni como variable (no terminal) ni como terminal, el validador lo identifica y lo reporta de inmediato mediante los códigos VAL004, VAL005 y VAL006, según si el símbolo aparece como cuerpo único de la producción o como primer o segundo símbolo de un cuerpo de dos. De esta forma, cualquier variable "inútil" en el sentido de estar mal referenciada o de no pertenecer realmente al alfabeto declarado de la gramática queda detectada antes de iniciar la conversión, evitando que el algoritmo intente procesar una gramática con símbolos ambiguos o inexistentes. Esta validación se suma a la de variables inalcanzables (VAL010, caso 5), de modo que el sistema cubre tanto los símbolos que no están correctamente declarados como los que, estando bien declarados, nunca participan en una derivación desde el símbolo inicial.

Finalmente, trabajar con una arquitectura desacoplada (backend en Python/Flask, frontend en React/Vite, desplegados en plataformas independientes) permitió al equipo experimentar un flujo de trabajo cercano al de un desarrollo de software real, incluyendo la configuración de CORS, variables de entorno para la comunicación entre servicios, y la necesidad de diseñar la API pensando en un consumidor desacoplado del servidor. Esta experiencia, sumada a la resolución del problema de rendimiento documentado en el capítulo de Implementación, consolidó en el equipo no solo el dominio teórico de la Forma Normal de Greibach, sino también competencias prácticas de ingeniería de software aplicadas a un problema de teoría de la computación.

10. REFERENCIAS BIBLIOGRÁFICAS

Baeldung. (2023). *Kosaraju's algorithm for strongly connected components*. Baeldung on Computer Science. https://www.baeldung.com/cs/kosaraju-algorithm-scc

Flask. (s.f.). *Welcome to Flask — Flask Documentation (3.1.x)*. Pallets Projects. https://flask.palletsprojects.com/en/stable/

Greibach, S. A. (1965). A new normal-form theorem for context-free phrase structure grammars. *Journal of the ACM, 12*(1), 42–52. https://dl.acm.org/doi/10.1145/321250.321254

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3.ª ed.). Addison-Wesley. https://www.abebooks.com/Introduction-Automata-Theory-Languages-Computation-Hopcroft/32298852295/bd

React. (s.f.). *React – The library for web and native user interfaces*. Meta Open Source. https://react.dev/

Sipser, M. (2012). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning. https://www-users.york.ac.uk/~ss44/books/pages/s/MichaelSipser.htm

Tailwind CSS. (s.f.). *Tailwind CSS — Rapidly build modern websites without ever leaving your HTML*. https://tailwindcss.com/

Vite. (s.f.). *Vite — Next generation frontend tooling*. https://vite.dev/

Wikipedia. (2024, 15 de marzo). *Chomsky normal form*. En *Wikipedia, la enciclopedia libre*. https://en.wikipedia.org/wiki/Chomsky_normal_form

Wikipedia. (2024, 15 de marzo). *Greibach normal form*. En *Wikipedia, la enciclopedia libre*. https://en.wikipedia.org/wiki/Greibach_normal_form